arXiv:2606.17315v1 [cs.DC] 15 Jun 2026

Space-Efficient Lock-Free Linear-Probing Hash Table

Hagit Attiya
Technion, Israel
hagit@cs.technion.ac.il

Rotem Oshman
Tel-Aviv University, Israel
roshman@tau.ac.il

Noa Schiller
Tel-Aviv University, Israel
noaschiller@mail.tau.ac.il

Abstract

Linear probing is one of the simplest and most space-efficient approaches to hash table design, and is widely used in sequential settings due to its compact memory layout. However, designing a *concurrent* linear-probing hash table with strong liveness guarantees has proved difficult, and only a handful of such algorithms have been proposed, all of which either restrict concurrency or rely on large per-entry metadata, thereby compromising space efficiency.

We present a lock-free linear-probing hash table with wait-free lookups that retains the core advantages of sequential linear probing while handling contention gracefully. Our design uses only a small amount of metadata per table entry: a constant number of additional bits when using LL/SC, or a logarithmic number of bits when using CAS. The algorithm is linearizable and lock-free, supports insert, delete, and wait-free lookup operations, and is able to safely reclaim space used by deleted elements without rebuilding the table.

We analyze the amortized step complexity of our hash table assuming no concurrent insertions of the same key, and show that each operation has expected amortized step complexity matching that of sequential linear probing, up to the point contention per key.

Contents

1 Introduction **2**

2 Additional Discussion of Related Work **3**

3 Preliminaries **5**

4 The Hash Table Construction **6**
4.1 High-Level Overview 6
4.2 Detailed Description 8
4.3 Insert Operations that Return ABORT 13

5 Correctness Proof **14**
5.1 Basic Properties 14
5.2 Linearizability 16
5.3 Liveness 26
5.4 Amortized Step Complexity 28

6 Summary and Discussion **30**

1 Introduction

Linear probing is an approach to hash table design where the table consists of a contiguous array of memory cells, and each key v is stored either in its hash location $h(v)$ or, if this location is taken, in the first free cell following $h(v)$. To search for key v, we start at location $h(v)$ and scan forward, until we either find v or we reach an empty cell. This simple design avoids pointer indirection and benefits from contiguous memory layout. When the parameters are tuned properly, a typical operation may access only a single cache line.

While there is a vast literature on *sequential* linear probing hash tables, only a few *concurrent* hash tables are based on linear probing [3, 6, 7, 14, 18], or indeed, on any form of *open addressing*, where the hash table is one contiguous array, and any key can be stored in any memory location, so that the hash table is "full" only when the entire array is full. This may be due to the fact that implementing a lock-free linear-probing hash table in the concurrent setting is much more challenging than in the sequential setting. To our knowledge, virtually all concurrent hash tables proposed so far either use pointers and indirection (e.g., [8, 15, 17, 19]), rely on locks to coordinate concurrent relocation of keys (e.g., [9]), limit concurrency (e.g., [20]), require frequent rebuilding (e.g., [7, 14]), and/or use heavyweight concurrency control mechanisms that make them space-inefficient (e.g., [18]).

Much of the appeal of linear probing comes from its compact, contiguous memory layout: accessing the hash table requires no indirection and exploits the structure of modern memory hierarchies. Storing large descriptors, timestamps, or auxiliary pointers in each cell undermines these advantages by increasing space usage and memory traffic. In this paper we present the first lock-free linear-probing hash table that uses *bounded per-cell metadata*, and safely reuses memory previously used by deleted elements. Our hash table enjoys nearly all the benefits of sequential linear probing while handling contention gracefully, and we prove that its expected amortized step complexity nearly matches that of *sequential* linear probing.

We give two versions of our hash table: one using the LL/SC synchronization primitive and constant metadata per hash table cell, and one using CAS and logarithmic metadata per cell. Let U be the size of the key domain, n be the number of processes, and m be the size of the hash table (i.e., the maximum number of keys that may be in the dictionary simultaneously); our main result is the following:

Theorem 1. *There is a lock-free linearizable concurrent linear-probing hash table with wait-free* lookup*s, where each memory cell stores* $\lceil \log U \rceil + O(1)$ *bits when using the* LL/SC *primitive; or* $\lceil \log U \rceil + \min\{\lceil \log m \rceil, \lceil \log n \rceil\} + O(1)$ *bits when using the* CAS *primitive.*

We analyze the expected amortized runtime of operations in our hash table when there are no concurrent insertions of the same key. This assumption is used only to simplify the runtime analysis, and is not needed for linearizability or lock-freedom. We show that in any execution that includes at most $(1 - 1/x)m$ insertions, with no two insertions of the same key overlapping in time and at most c operations on any given key overlapping in time, the expected amortized runtime of each operation is $O(x^2 + c)$. This matches the classical analysis of linear probing due to Knuth [12], up to an additive *point contention* term of c.

To our knowledge, ours is the first lock-free linear-probing hash table that uses bounded memory cells and does not require rebuilding the table, as long as the number of cells occupied by keys does not exceed the size of the table. Concurrent insertions of the same key may temporarily take up

space in the hash table, but this space is made available for reuse immediately upon the return of the redundant insert operations.

Using tombstones in concurrent linear-probing hash tables. A key design choice in implementing a linear-probing hash table is how to handle deletions. One option is to shift elements to fill the resulting hole; however, shifting is difficult to implement correctly under concurrency, and synchronizing it often requires substantial metadata (for example, in [3]). Another option is to use *tombstones*, a special symbol that marks cells previously occupied by a deleted element, so that lookups continue traversing past them. However, tombstones may accumulate and degrade performance unless the table is periodically rebuilt (as in, e.g., [7, 14]).

Our hash table uses tombstones, but it allows inserts to *reuse* tombstones, effectively treating them as empty cells where new keys can be inserted. This introduces a new challenge: concurrent inserts of the same key may observe different tombstones and temporarily create multiple copies of the same key in the table (we give an example of this in Figure 2 below). Our main algorithmic contribution is a mechanism that resolves such conflicts using only bounded metadata per cell.

In our hash table, operations perform steps only "on their own behalf" and do not attempt to help other operations to complete. Rather than ensuring that other inserts of the same key complete, operations are allowed to withdraw and return before ascertaining which copy is the "final winner". This helps us avoid large metadata, but it requires delicate synchronization between concurrent operations on the same key. For example, a challenging aspect of our design is that an ultimately *unsuccessful* insert operation may force the key to be treated as present by subsequent operations, even though there has not been a successful insert yet. The linearizability proof addresses this challenge by introducing the notion of *insert sequences*, defined as chains of insert operations where the tentative copy of one operation is eliminated by the next in the sequence. The final copy in the sequence is successful (i.e., returns *true*). Even with this notion in hand, ordering insert operations is subtle, because there can be several insert sequences *on the same key* that overlap in time. In particular, *unsuccessful* insert operations are not necessarily ordered after the "final winner" in their own insert sequence.

2 Additional Discussion of Related Work

Concurrent hash tables have been studied extensively, with many designs targeting scalability, non-blocking progress, and dynamic resizing. Most modern concurrent hash tables rely on chaining, bucket-based layouts, or other forms of indirection, often combined with fine-grained locking or lock-free pointer-based synchronization (e.g., [8, 15, 19]).

Only a few concurrent hash tables exist that use linear probing, since open addressing severely constrains synchronization. Table 1 compares our implementations and previous work. Next we briefly discuss the implementations that appear in the table.

Purcell and Harris [18] present one of the earliest lock-free open-addressing hash tables. Their algorithm uses quadratic probing and, instead of marking deletions with tombstones, stores an upper bound on the probe length at each cell. Their use of quadratic probing appears orthogonal to the synchronization mechanisms used to manage duplicate copies.

Like ours, the algorithm of [18] allows multiple copies of the same key to coexist temporarily; however, it resolves conflicts using helping together with rich per-cell metadata. Duplicate copies are eliminated dynamically based on probe order: when an insertion encounters an earlier copy, it

	Memory Primitive	Deletion Method	Cell Reuse?	Cell Size
[18]	CAS	Probe Bounding	✓	Unbounded
[7, 14]	CAS	Tombstones	✗	$\lceil \log U + 2 \rceil$
[3]	Improper LL/SC	Shifting	✓	$2\lceil \log U + 1 \rceil + 2$
[20]	CAS	Shifting *(limited concurrency)*	✓	$\lceil \log U + 1 \rceil$
This paper	**Proper** LL/SC	**Tombstones**	✓	$\boldsymbol{\lceil \log U + 1 \rceil + 2}$
This paper	CAS	**Tombstones**	✓	$\boldsymbol{\lceil \log U + 1 \rceil + \lceil \log n \rceil + 2}$

Table 1: Comparison of concurrent linear probing implementations, detailing the synchronization primitive used for each one (CAS, improperly-paired LL/SC, or properly-paired LL/SC), the deletion method, whether or not the hash table is able to re-use cells after deletion, and the size of each memory cell in terms of the key domain size U. Cell sizes are reported for the core data structure and do not include auxiliary space used for resizing.

withdraws its own copy and helps complete the earlier one. As a result, insert operations return only after ensuring that some copy has been finalized. This helping mechanism relies on unbounded timestamps to avoid the ABA problem.

Removing timestamps completely from the hash table of [18] appears difficult due to the requirements of the helping mechanism. In particular, because helping may require tracking and finalizing *multiple* tentative copies simultaneously, a naïve attempt to replace timestamps by LL/SC would result in *improperly-paired* LL/SC reservations, where several LL reservations are simultaneously pending. This is incompatible with current architectures that *allow only one active* LL *reservation at a time*. (For example, the ARM architecture explicitly prohibits mismatched addresses between its LL/SC equivalents, and violating this results in unpredictable behavior [2, Section B2.12.5].) One might also think of using process identifiers to help manage the ABA problem instead of using timestamps, but this makes it difficult to distinguish between old and new copies created by the same process.

In other lock-free open-addressing implementations that use tombstones to mark deletions [7,14], tombstones are not reused (or are reused only for the same key), preventing duplicate copies from arising during overlapping insertions of the same key. This restriction simplifies synchronization and eliminates the need for per-entry metadata, but causes tombstones to accumulate over time, increasing table occupancy beyond the number of stored keys and requiring periodic rebuilding. Our construction is the first lock-free linear-probing hash table to combine tombstone reuse with bounded per-cell metadata. We believe that the dynamic resizing mechanism integrated in these concurrent open-addressing hash tables can be used with our hash table.

As mentioned in Section 1, the alternative to tombstones is to shift keys backwards to take up the space freed up by deleted keys. Concurrent linear probing with backward-shift deletion is significantly harder as it requires updating multiple cells in the hash table. In [3], a concurrent Robin Hood hash table uses improperly-paired LL/SC to emulate two-word synchronization primitives; this hash table also stores two keys in each cell, halving its capacity. Shun and Blelloch [20] implement backward-shift deletion in the *phase-concurrent* setting, where inserts and deletes are

not allowed to overlap.

While a CAS-based version of our hash table can be obtained by using a general-purpose simulation to replace LL/SC by CAS, this would incur high memory overhead: these simulations either increase the size of each memory cell by at least n bits [10, 16], or utilize auxiliary memory arrays, thereby reducing data locality [1, 5, 11]. In contrast, the CAS-based version of our hash table requires only $\lceil \log n \rceil$ additional bits per cell compared to the LL/SC version.

3 Preliminaries

We use the standard asynchronous shared memory model, in which n processes communicate through shared *memory cells*. The shared memory is accessed by executing *primitive operations*. The LL/SC primitive supports the following operations: LL(x) returns the value stored in memory cell x, and SC(x, *new*) writes the value *new* to x, if it was not written since the last time the process performed an LL(x) operation (otherwise, x is not modified). SC returns *true* if it writes successfully, and *false* otherwise. LL/SC operations should be *properly-paired*: an LL(x) is eventually followed by SC(x, _) on the same memory cell x, without an intervening LL or SC to a different memory cell. The CAS primitive supports the operation CAS(x, *old*, *new*), which atomically checks if the current value of the memory cell x is *old*, and if so, replaces it by *new* and returns *true*; otherwise, the operation leaves the value unchanged, and returns *false*. Memory cells also support read operations.

An implementation of an abstract data type specifies, for each process, a program for each operation; when receiving an *invocation* of an operation, the process takes *steps* according to its program. Each step by a process consists of some local computation, followed by a single primitive operation on the shared memory. The process may change its local state after a step, and it may return a *response* to the operation.

A *configuration* C specifies the state of every process and every memory cell. An *execution* α is an alternating (finite or infinite) sequence of configurations and steps, starting with a configuration. Operation o_1 *precedes* operation o_2 in α, if o_1's response precedes o_2's invocation in α; in this case, we also say that o_2 *follows* o_1.

An execution α induces a *history* $H(\alpha)$, consisting only of the invocations and responses of higher-level operations. An invocation *matches* a response if they both belong to the same operation. An operation *completes* in H if H includes both the invocation and response of the operation; if H includes the invocation of an operation, but no matching response, then the operation is *pending*. A history H is *sequential* if every invocation is immediately followed by a matching response.

A *completion* of history H is a history H' whose prefix is identical to H, and whose suffix includes zero or more responses of pending operations in H. Let comp(H) be the set of all of H's completions. A sequential history H' is a *linearization* of an execution α if (1) H' is a permutation of a history in comp($H(\alpha)$), (2) H' matches the sequential specification of the implemented object, and (3) H' respects the real-time order of non-overlapping operations in $H(\alpha)$. An execution α is *linearizable* if it has a linearization, and an implementation of an abstract object is linearizable if all of its executions are linearizable.

An implementation is *lock-free* if whenever there is a pending operation, some operation returns in a finite number of steps of all processes; it is *wait-free* if whenever there is a pending operation by process p, this operation returns in a finite number of steps by p.

4 The Hash Table Construction

A hash table implements a *dictionary*, which represents an unordered set of elements, and supports the operations insert(v), delete(v) and lookup(v). Each operation takes a *key* v and returns *true* or *false*: lookup(v) and delete(v) return *true* if v is in the set, while insert(v) returns *true* if v is not in the set. An insert may return a special value, ABORT, indicating that no available slot was found to insert the key (i.e., the hash table is "full"). Aborts are expected to be a rare event and they should trigger a rebuild of the hash table using standard techniques, which we do not treat here (see Section 4.3 for further discussion). ABORTs do not affect the logical state of the hash table; in terms of the sequential semantics, we model them by allowing insert operations to nondeterministically return ABORT at any point, without modifying the table.

4.1 High-Level Overview

The main challenge in a concurrent linear-probing hash table that reuses tombstones is handling overlapping insertions of the same key, because such insertions may temporarily create multiple copies of the key in the table. We must handle these copies in a way that ensures that *exactly one copy* eventually survives (barring deletion of the key).

The key invariant underlying the algorithm is that surviving copies of a key can move only toward earlier positions in the probe sequence beginning at $h(v)$. Intuitively, among multiple copies of a key v, copies that are closer to $h(v)$ have priority over later copies. This probe-order priority prevents cycles of competing eliminations. A similar approach was taken in [18], but here we avoid helping and large metadata.

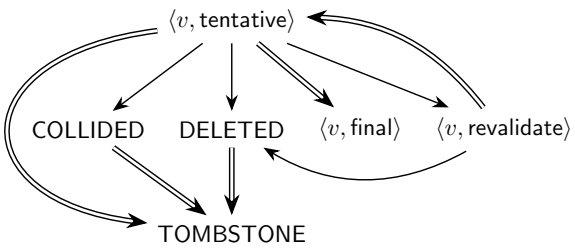
To realize the invariant, an insert operation first places a *tentative copy* of the key in the table. Tentative copies may subsequently be finalized, withdrawn, or eliminated by competing operations. Operations searching for the key treat tentative copies as logically present, even before the surviving copy is determined. This behavior is essential for linearizability, and is formalized later using the notion of *insert sequences* (see Section 5.2.2).

We focus first on the version of the algorithm that uses the LL/SC primitive, in which each memory cell stores a key together with only a constant number of tag bits. At the end of Section 4.2 we show how to modify the hash table to use CAS, by storing an index of a cell or a process within each cell (an additional $O(\min(\log m, \log n))$ bits).

Using metadata to handle duplicates. We use constant-size metadata to manage the competition between overlapping inserts, as well as the interaction with concurrent delete and lookup operations on the same key. A key v is first inserted into the table as a *tentative copy*, $\langle v, \mathsf{tentative}\rangle$. The tentative copy reserves the cell for the inserting operation, although other operations may still modify the metadata in the cell in order to communicate with the inserting process. There are several ways the insertion can proceed from here (see Figure 1):

- The insertion can be finalized by having the original insert operation write $\langle v, \mathsf{final}\rangle$.
- The tentative copy can be withdrawn by writing TOMBSTONE into the cell. Only the original inserting operation can do this, either upon request from another operation (described next) or upon finding another copy of the same key closer to location $h(v)$.

Figure 1: The life cycle of a tentative copy of key v. Double lines indicate changes that can only be made by the "owner", i.e., the insert operation that created the tentative copy. In addition, the value COLLIDED is written only by a concurrent insert(v) operation, and DELETED only by a concurrent delete(v) operation.



- A different insert(v) or delete(v) operation can *eliminate* a copy of v by writing COLLIDED or DELETED, respectively. This will cause the original inserting operation to clean up the cell and convert it into a TOMBSTONE. The value written to the cell determines the response of the inserting operation: *true* for DELETED and *false* for COLLIDED.
- A different process carrying out lookup(v) or insert(v) can also signal the inserting operation that it has *seen* the tentative copy and "relied on it", by writing $\langle v, \text{revalidate}\rangle$. This causes the original inserting operation to stop trying to withdraw its tentative copy if it is currently doing so, and to revalidate it by checking again for duplicates (see below).

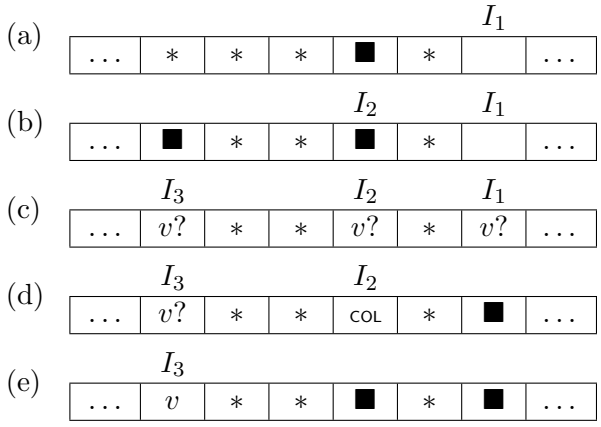
Validating a tentative copy by checking for duplicates. After writing a tentative copy of its key into some location i, an insert operation re-scans from location $h(v)$ to the end of the run (the maximal contiguous sequence of non-EMPTY cells starting from $h(v)$), searching for other copies of v. If a competing copy of v is found *before* location i (i.e., between $h(v)$ and i), then the other copy "wins", and the insert operation attempts to withdraw its tentative copy and convert it into a tombstone. This may fail if another process signals that it has seen the tentative copy, causing the operation to restart the validation process. If a competing tentative copy of v is found in some location j *after* location i, then the copy in the earlier location $i < j$ "wins", and the inserting process *eliminates* the copy in location j by writing COLLIDED into location j (see more on this process below). This, too, may fail — if the copy in location j is already finalized before the value COLLIDED can be written, then location j becomes the final copy, and the insert operation for location i will give up and try to delete its own tentative copy.

Although we refer to them as *tentative copies*, if there is a tentative copy of key v in the table, all later operations must behave as though v is logically present in the table. The "tentative" aspect is only regarding *which copy will win*, not regarding *whether v is inserted*.

Because each tentative copy can only (try to) eliminate other tentative copies that precede it in the hash table, a chain of attempted inserts of the same key v that eliminate each other goes *backwards* through the hash table, towards location $h(v)$. We refer to such chains as *insert sequences*, and the fact that surviving copies can only move "up the table" towards $h(v)$ is crucial to the correctness of the algorithm. Figure 2 illustrates how a copy moves towards $h(v)$.

Safely eliminating tentative copies. A critical part of the algorithm is the safe elimination of later tentative copies of the same key. Before eliminating another copy of v, an insert(v) operation must ensure that its own tentative copy has not been concurrently deleted: without this check, an insert that already "lost" (i.e., been deleted by a concurrent delete(v) operation, or had its copy eliminated by a concurrent insert(v)) could erroneously eliminate tentative copies belonging

Figure 2: Three concurrent insertions, I_1, I_2 and I_3, of the same key v; cells occupied by keys other than v hold $*$. (a) I_1 prepares to write a tentative copy; deletion creates a tombstone. (b) I_2 finds a tombstone and prepares to write a tentative copy; deletion creates another tombstone. (c) I_3 finds a tombstone; all three operations write tentative copies (v?). (d) I_1 sees copy written by I_3 and withdraws; I_3 sees copy written by I_2 and eliminates it by writing COLLIDED. (e) I_2 cleans up by writing a tombstone; I_3 promotes its copy to final.



to future insertions that began after it had already lost. This could lead to *no copies of v surviving*, even though "logically", v has been re-inserted into the table after being deleted.

Why revalidation is required. As we said above, a lookup(v) or insert(v) operation that encounters a tentative copy of v will signal the inserting operation to revalidate, i.e., stop withdrawing the copy if it is currently doing so, and check again for duplicates. Subsequently, the lookup or insert operation treats the tentative copy of v as though it were finalized — lookup(v) returns *true*, and insert(v) returns *false*.

Requesting revalidation is necessary because the lookup or insert operation has no way of knowing whether the tentative copy that it has seen is about to be withdrawn. For example, suppose operation I = insert(v) has already written its tentative copy into location i, but then found another tentative copy in location $h(v) < j < i$ and decided to withdraw its copy in location i. Before I can write a tombstone to location i, the preceding copy of v in location j is deleted. Then a lookup(v) or insert(v) operation O begins scanning forward from $h(v)$ and finds the copy of v in location i, even though the key is no longer logically in the set. Operation O has no way of knowing whether operation I is about to finalize its copy or withdraw it, and does not wait around to see; instead, it treats the key as though it is bound to be finalized, and signals to I that it has done so by requesting revalidation. This "resurrects" the key in location i and causes I to resume trying to finalize it.

4.2 Detailed Description

We now describe how these ideas are realized in the code. We provide two versions of the algorithm. The first uses the LL/SC primitive to safely verify and delete tentative copies. The second uses the more common CAS primitive, emulating the required behavior with a small amount of additional per-cell metadata. Both implementations have the same high-level structure, and we encapsulate the use of the two different synchronization primitives by using a few primitive-specific functions (Algorithm 5 for LL/SC and Algorithm 6 for CAS) and by using the keywords Read and Modify in the code. When using LL/SC, the keyword Read is an invocation of LL, and Modify is an SC; in the CAS version, Read is standard atomic read, and Modify is an invocation of CAS.

We use modular arithmetic that wraps at the m-th cell, and omit the notation (mod m).

Data structures. The table is a single array table$[0 \dots m-1]$, where each cell stores either a tagged key $\langle v, tag \rangle$ where $tag \in \{\mathsf{tentative}, \mathsf{final}, \mathsf{revalidate}\}$, or one of the values EMPTY, TOMBSTONE representing an available cell, or one of a few internal cleanup states (DELETED, COLLIDED, and in the CAS version, $\langle \langle v, j \rangle, \mathsf{marked} \rangle$, where j is the index of a cell). Using the two tag bits, one reserved sentinel value is sufficient to encode the four states that do not contain a key. Thus, these values can be encoded using $\lceil \log(U+1) \rceil + 2$ bits when using LL/SC, and $\lceil \log(U+1) \rceil + \min(\lceil \log n \rceil, \lceil \log m \rceil) + 2$ bits for CAS. In the code, the function $\mathsf{val}(x)$ extracts the key from encoded cell x (or $\perp$), and $\mathsf{restart}(x)$ checks whether the owner of a tentative copy should validate it, which holds if $x = \langle v, \mathsf{revalidate} \rangle$ or $x = \langle \langle v, * \rangle, \mathsf{marked} \rangle$ (with CAS).

Searching for a key. All operations begin by searching for an existing copy of key v. First, operations execute a *forward scan*, which begins at location $h(v)$ and scans forward until it either finds v in the hash table or reaches the end of the run (an EMPTY cell). If the key was not found during the forward scan, operations then execute a *backward scan* from the end of the run back to $h(v)$. Since a sequence of tentative copies that eliminate one another moves back through the table towards location $h(v)$, a backward scan ensures that at least one of these copies is eventually encountered, provided at least one remains in the table.

At any point during the forward and the backward scan, if key v is found, then action may be taken, depending on the type of the operation: a $\mathsf{delete}(v)$ operation will try to delete the key, and a $\mathsf{lookup}(v)$ or $\mathsf{insert}(v)$ operation return *true* or *false*, respectively. We remark that "finding key v" does not merely amount to finding a cell where a copy of key v is stored: if the copy that is found is not finalized (i.e., not $\langle v, \mathsf{final} \rangle$) and the searching operation is $\mathsf{lookup}(v)$ or $\mathsf{insert}(v)$, then the searching operation will request that the original inserting operation revalidate its copy, by writing $\langle v, \mathsf{revalidate} \rangle$ into the cell (if no other process already requested this).

Operations $\mathsf{delete}(v)$ and $\mathsf{lookup}(v)$ return after completing their forward and backward scan, but for $\mathsf{insert}(v)$ the work has only begun. At the end of the backward scan, upon returning to location $h(v)$, an $\mathsf{insert}(v)$ operation probes forward until it finds an available cell, either EMPTY or TOMBSTONE. It then attempts to insert a tentative copy of key v by writing $\langle v, \mathsf{tentative} \rangle$, and if successful, checks for duplicates (as explained above).

Scanning. The forward scan and the backward scan are detailed in Algorithm 1. The forward scan from $h(v)$ proceeds until encountering EMPTY or completing a full cycle. At each cell that contains the key v, the scan invokes either validate_copy (for insert/lookup) or try_delete (for delete); these helper functions are described below, but intuitively, validate_copy attempts to confirm the presence of a "stable copy" of the key, and returns *true* if it succeeds; try_delete attempts to convert the key into a tombstone or signal the inserting process to do so, returning a non-$\perp$ value if it succeeds. If the forward scan sees no decisive result, it returns the index of the last non-empty cell in v's probe run. The backward scan then moves backwards toward $h(v)$ and searches for v, again invoking the appropriate helper functions.

Lookup and delete. Lookup and delete operations (Algorithm 2) both perform a forward scan and, if needed, a backward scan. Upon finding a copy of v, they invoke a helper function (validate_copy or try_delete, respectively; see Algorithm 4 below), and if that helper function returns *true*, the operation returns *true*.

Algorithm 1: forward and backward scan

```
1  function forward_scan(v, op_type):
2      i ← h(v) ; val ← Read(table[i])
3      while val ≠ EMPTY do
4          if val(val) = v then
5              if op_type ∈ {insert, lookup} and validate_copy(v, val, i) then return true
6              if op_type = delete then
7                  rsp ← try_delete(v, val, i)
8                  if rsp ≠ ⊥ then return rsp
9          i ← i + 1
10         if i = h(v) then break                          // Scanned all cells in the table
11         val ← Read(table[i])
12     if i = h(v) and val = EMPTY then return i                          // An empty run
13     else return i − 1                          // Go back one cell to the end of the run
14 function backward_scan(v, i, op_type):
15     while true do
16         val ← Read(table[i])
17         if val(val) = v then
18             if op_type ∈ {insert, lookup} and validate_copy(v, val, i) then return true
19             if op_type = delete then
20                 rsp ← try_delete(v, val, i)
21                 if rsp ≠ ⊥ then return rsp
22         if i = h(v) then return ⊥                                  // Back to the start
23         i ← i − 1
```

Algorithm 2: delete and lookup

```
24 function delete(v):
25     rsp ← forward_scan(v, delete)
26     if rsp ∈ {true, false} then return rsp
27     rsp ← backward_scan(v, rsp, delete)
28     if rsp ∈ {true, false} then return rsp
29     return false

30 function lookup(v):
31     rsp ← forward_scan(v, lookup)
32     if rsp = true then return true
33     rsp ← backward_scan(v, rsp, lookup)
34     if rsp = true then return true
35     return false
```

Insert. An insert (Algorithm 3) first checks for an existing copy via the forward and backward scans; if found (using the same criteria as a successful lookup), it returns *false*. Otherwise, it probes for an EMPTY/TOMBSTONE cell and attempts to write $\langle v, \text{tentative} \rangle$. It then rescans the run of v: encountering an earlier or already inserted copy causes the operation to delete its own tentative copy using del_copy and abort. If del_copy returns $\perp$, this means another operation requested revalidation, so the key must be "resurrected" and revalidated. Encountering a later tentative copy triggers del_other_copy, which attempts to eliminate it (returning failure if this attempt is unsuccessful). This attempt can fail if the later copy was already promoted to final, if the eliminating process's own tentative copy has been marked as deleted, or if another process has requested revalidation. Upon reaching EMPTY, the operation tries to promote its tentative copy to $\langle v, \text{final} \rangle$. If its tentative copy has been marked revalidate, it repeats the validation; if marked COLLIDED or DELETED, it cleans up and returns *false* or *true*, respectively.

Algorithm 3: insert

```
36 function insert(v):
37     rsp ← forward_scan(v, insert)
38     if rsp = true then return false
39     rsp ← backward_scan(v, rsp, insert)
40     if rsp = true then return false
41     j ← h(v) ; val ← Read(table[j])
42     while true do                                              // Try to insert v
43         if val ∈ {EMPTY, TOMBSTONE} and Modify(table[j], val, ⟨v, tentative⟩) then
44             break
45         j ← j + 1
46         if j = h(v) then return ABORT                  // Scanned all cells in the table
47         val ← Read(table[j])
       // search for duplicate copies
48     i ← h(v) ; val ← Read(table[i])
49     while val ≠ EMPTY do
50         if i ≠ j and val(val) = v then                 // Skip own copy and cells without v
51             if i − h(v) < j − h(v) or val = ⟨v, final⟩ then   // Other copy is earlier or final
52                 rsp ← del_copy(v, j)
53                 if rsp = ⊥ then goto Line 48                   // Revalidate copy
54                 return rsp
55             else if val ≠ ⟨v, revalidate⟩ then
56                 if ¬del_other_copy(v, val, i, j) then          // Try to eliminate other copy
57                     cur_val ← Read(table[j])
58                     if restart(cur_val) and Modify(table[j], cur_val, ⟨v, tentative⟩) then
59                         goto Line 48                           // Revalidate copy
60                     rsp ← del_copy(v, j)
61                     if rsp = ⊥ then goto Line 48               // Revalidate copy
62                     return rsp
63         i ← i + 1
64         if i = h(v) then break                     // Scanned all cells in the table
65         val ← Read(table[i])
66     cur_val ← Read(table[j])
67     if cur_val = ⟨v, tentative⟩ and Modify(table[j], cur_val, ⟨v, final⟩) then
           // Try to fully insert own copy
68         return true
69     if restart(cur_val) and Modify(table[j], cur_val, ⟨v, tentative⟩) then
70         goto Line 48                                           // Revalidate copy
71     rsp ← del_copy(v, j)
72     if rsp = ⊥ then goto Line 48                               // Revalidate copy
73     return rsp
```

Helper functions. The function validate_copy (used by lookup and insert) returns *true* if it identifies a finalized copy of v, marks a copy of v for revalidation (or sees an existing request), or observes that a copy of v was rewritten; it returns *false* if re-reading the cell reveals that the copy of v is no longer present. If neither scan finds a copy of v that is confirmed by validate_copy, the lookup returns *false*. A delete uses try_delete, which transforms $\langle v, \text{final}\rangle$ into TOMBSTONE, and overwrites a tentative copy with DELETED, to request that their owner clean up. The function

Algorithm 4: Helper functions

```
74 function del_copy(v, j):                                  // Deletes own copy and cleans the cell
75     repeat
76         val ← Read(table[j])
77         if val = ⟨v, revalidate⟩ then
78             if Modify(table[j], val, ⟨v, tentative⟩) then return ⊥
79             val ← Read(table[j])
80     until Modify(table[j], val, TOMBSTONE)
81     if val = DELETED then return true
82     return false
83 function try_delete(v, val, i):                  // If the cell contains the key try to delete it
84     while val(val) = v do
85         if val = ⟨v, final⟩ then
86             return Modify(table[i], val, TOMBSTONE)
87         else
88             if Modify(table[i], val, DELETED) then return true
89         val ← Read(table[i])
90     return ⊥
91 function validate_copy(v, val, i):              // Detects v and requests revalidation if needed
92     if val ∈ {⟨v, final⟩, ⟨v, revalidate⟩} then return true
93     if Modify(table[i], val, ⟨v, revalidate⟩) then return true
94     val ← table[i]
95     if val(val) = v then return true
96     return false
```

Algorithm 5: LL/SC-specific code

```
97 function Read(loc):
98     return LL(loc)
99 function Modify(loc, old, new):
100    return SC(loc, new)
101 function del_other_copy(v, val, i, j):
102    cur_val ← table[j]
103    if cur_val = ⟨v, tentative⟩ then
104        if ¬SC(table[i], COLLIDED) then
105            val ← table[i]
106            if val = ⟨v, final⟩ then return false
107        return true
108    return false
```

try_delete returns *true* if it successfully deletes the key, *false* if it observes that another process deleted a finalized key (in which case the delete operation can also return *false*), and $\perp$ if a copy of v is no longer present. If neither the forward nor the backward scan deleted anything, the delete operation returns *false*.

Algorithm 6: CAS-specific code

```
109 function Read(loc):
110 |   return *loc
111 function Modify(loc, old, new):
112 |   return CAS(loc, old, new)
113 function del_other_copy(v, val, i, j):
114 |   if val = ⟨⟨v, k⟩, marked⟩ and k ≠ j then return true   // Reserved by a different insert
115 |   if val ≠ ⟨⟨v, j⟩, marked⟩ then                          // Try to reserve the copy
116 |   |   if ¬CAS(table[i], val, ⟨⟨v, j⟩, marked⟩) then
117 |   |   |   val ← table[i]
118 |   |   |   if val = ⟨v, final⟩ then return false
119 |   |   |   return true
120 |   cur_val ← table[j]
121 |   if cur_val = ⟨v, tentative⟩ then                       // Try to fully delete other copy
122 |   |   CAS(table[i], ⟨⟨v, j⟩, marked⟩, COLLIDED)
123 |   |   return true
124 |   return false
```

LL/SC and CAS versions. Beyond replacing the keywords Read and Modify by LL/SC or by atomic read and CAS (respectively) as explained above, the part of the algorithm that eliminates duplicates is slightly different for the two primitives. In the LL/SC version (Algorithm 5), before writing over the losing copy using an SC, the process checks that its own copy has not been eliminated between the LL and the SC to the other copy. This ensures the deleting process's copy was present in the table at the same time as the one eliminated. If this SC fails, the process checks if the other copy has been finalized, and if so, it gives up on its own copy. If the other copy has not been finalized, the process continues as though it were able to write COLLIDED, safe in the knowledge that some other process has handled this copy or deleted it.

It can be verified that our algorithm uses LL/SC in a *properly-paired manner*, where an SC targets the same cell as the preceding LL, avoiding nested or overlapping patterns that are usually not supported on current architectures.

In the CAS version (Algorithm 6), duplicate elimination uses a temporary marked state $\langle\langle v, j\rangle, \mathsf{marked}\rangle$, where j is the index of the cell locked by the operation attempting to delete this copy. The eliminating process uses this state to claim *provisional ownership* of the remote tentative copy before converting it to COLLIDED, emulating LL/SC-style reservation using only CAS in the specific context of our algorithm (this is not a general emulation of LL/SC from CAS). This adds $O(\log m)$ bits; however, if $\log n < \log m$ we can instead use process identifiers. Because the process executing the insert effectively "locks" the cell for the duration of the operation, process identifiers and cell indices can be used interchangeably to declare provisional ownership.

4.3 Insert Operations that Return ABORT

An insert operation that returns ABORT (in line 46 of Algorithm 3) has no effect on the abstract state of the dictionary. Inserts only abort if they complete a full scan of the hash table without finding an available cell (either EMPTY or TOMBSTONE).

An insert that aborts does not necessarily imply that the table is "logically full". This is because

insert operations of the same keys may temporarily take up space in the table by creating duplicate copies (which will be eliminated prior to their return), or concurrent delete operations may clear up space during the scan. However, insert operations do not abort spuriously when space is available: if an insert(v) operation starts running when the table has free space, and the table does not already contain key v, then some available cell must be used up by a key before the insert completes. This is formalized in the next proposition:

Proposition 2. *If an* insert(v) *operation is invoked at a configuration in which at least one cell in the table is empty or contains a tombstone, and no cell contains the key v, then some available cell in the table is reclaimed by a key before the operation returns.*

In particular, if the table contains a tombstone or an empty cell, then an insert operation running solo from a quiescent configuration will not abort. Aborts suggest that the hash table is full or nearly full. Repeated ABORT responses may therefore trigger a resize operation, which can also be initiated preemptively before the table becomes full.

5 Correctness Proof

In this section, we prove that the hash table is linearizable and lock-free, and analyze its amortized runtime complexity. Let U be the size of the key domain, n the number of processes, and m the size of the hash table.

Theorem 1. *There is a lock-free linearizable concurrent linear-probing hash table with wait-free* lookup*s, where each memory cell stores $\lceil \log U \rceil + O(1)$ bits when using the* LL/SC *primitive; or $\lceil \log U \rceil + \min\{\lceil \log m \rceil, \lceil \log n \rceil\} + O(1)$ bits when using the* CAS *primitive.*

5.1 Basic Properties

We begin by showing that the linear-probing structure is preserved and that, among concurrent insertions of the same key, at most one copy can be fully inserted. The proof abuses terminology and says that an operation performs a step when the process executing it performs the step.

A cell is *non-empty* if it does not contain EMPTY; a cell contains the key v if its value is one of $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{final}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$.

We observe that our algorithm preserves the following key property of sequential linear probing. This property implies that if a key is in the table, it will be found by scanning the table starting from the key's initial hash position until either the key is found or an empty cell is encountered. Moreover, two copies of the same key are part of the same run.

Proposition 3. *If a cell i contains the key v, then all cells j, $h(v) \leq j \leq i$, are not empty.*

Our algorithm ensures that, at any point in the execution, at most one insert operation can become the current *winner*, i.e., the one whose copy may be promoted to $\langle v, \mathsf{final}\rangle$ (formalized in Lemma 4 below). This invariant yields a well-defined ordering of successful insert and delete operations, whose linearization points are defined in Section 5.2.1.

Consider an insert(v) operation *op* that writes $\langle v, \mathsf{tentative}\rangle$ to a cell in the table. This value can be overwritten with $\langle v, \mathsf{final}\rangle$, DELETED, COLLIDED, TOMBSTONE or $\langle v, \mathsf{revalidate}\rangle$; in the CAS version it can also be overwritten with $\langle\langle v, *\rangle, \mathsf{marked}\rangle$. If it is overwritten with one of the first three

values, that value can be subsequently overwritten only with TOMBSTONE. If it is overwritten with $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$, then *op* may overwrite the value again with $\langle v, \mathsf{tentative}\rangle$.

Each value in the sequence $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ and $\langle\langle v, *\rangle, \mathsf{marked}\rangle$, starting from the first $\langle v, \mathsf{tentative}\rangle$ value written by *op*, is a *copy* of *op*. The sequence cycles through these states with restricted transitions: while $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ may transition to either $\langle v, \mathsf{tentative}\rangle$ or $\langle v, \mathsf{revalidate}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ is restricted to transition only back to $\langle v, \mathsf{tentative}\rangle$. The cell in which the copies of *op* are written is called *op*'s cell. A copy of *op* is *inserted* if it is overwritten with DELETED or $\langle v, \mathsf{final}\rangle$. If a copy is overwritten with $\langle v, \mathsf{final}\rangle$, we call it the *inserted copy* of operation *op*. A copy of *op* is *deleted* if it is overwritten with DELETED, COLLIDED or TOMBSTONE, or if the inserted copy of *op* is overwritten with TOMBSTONE. Note that only the last copy written by *op* can be inserted or deleted.

The next lemma formalizes a key invariant mentioned in Section 4.1, namely, that among all simultaneous attempts to insert duplicate copies of v, only one can be successful. Intuitively, this is because before succeeding, an insert scans the probe sequence and observes that no copy closer to the start of the probe sequence exists. In this scan, the insert ignores copies with the flag revalidate or marked. It can do so as the other insert operation must first rewrite the copy with the flag tentative and then scan the table, ensuring it will observe the current operation's copy.

Lemma 4. *At any point in the execution, for any key v, at most one cell in the table contains $\langle v, \mathsf{final}\rangle$.*

Proof. Assume, by way of contradiction, that two cells i and j, $i \neq j$, contain $\langle v, \mathsf{final}\rangle$ at the same point in α. Let op_i and op_j be the $\mathsf{insert}(v)$ operations that write these $\langle v, \mathsf{final}\rangle$ values to cells i and j, respectively. Assume, without loss of generality, that op_i writes $\langle v, \mathsf{final}\rangle$ to cell i before op_j writes $\langle v, \mathsf{final}\rangle$ to cell j.

An insert writes $\langle v, \mathsf{final}\rangle$ to a cell only after it writes $\langle v, \mathsf{tentative}\rangle$ to that cell. Consider the last time op_i writes $\langle v, \mathsf{tentative}\rangle$ to cell i, and note that it then scans a run starting from $h(v)$ before writing $\langle v, \mathsf{final}\rangle$ to cell i. We next assume the run includes cell j and consider the possible values op_i reads from cell j.

First, assume that op_i reads $\langle v, \mathsf{tentative}\rangle$ written by op_j from j. If cell j is closer to the initial hash than cell i (that is, $j - \mathsf{h}(v) < i - \mathsf{h}(v)$), then op_i deletes its own copy and never writes $\langle v, \mathsf{final}\rangle$ to cell i. Otherwise, op_i tries to delete the copy of op_j. Regardless of whether op_i succeeds in deleting the copy, the value in cell j changes. Since op_i did not write $\langle v, \mathsf{final}\rangle$ to cell i yet and is the first operation to write $\langle v, \mathsf{final}\rangle$ among the two, the value of cell j cannot change to $\langle v, \mathsf{final}\rangle$ and may only change to COLLIDED, DELETED, $\langle v, \mathsf{revalidate}\rangle$, or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$. The first two values are not possible, since cell j is eventually overwritten with $\langle v, \mathsf{final}\rangle$. Hence, either $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ are written in cell j after op_i reads cell j.

Otherwise, op_i reads from cell j either $\langle v, \mathsf{revalidate}\rangle$ copy of op_j, or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ copy of op_j, or the first write of $\langle v, \mathsf{tentative}\rangle$ to cell j by op_j follows the read of cell j by op_i. In all cases, the last write of $\langle v, \mathsf{tentative}\rangle$ to cell j by op_j (which exists since this last copy is overwritten with $\langle v, \mathsf{final}\rangle$) follows op_i's read of cell j.

If the run excludes cell j and op_i does not read cell j, by Proposition 3, the first time op_j writes $\langle v, \mathsf{tentative}\rangle$ to cell j follows the last time op_i writes $\langle v, \mathsf{tentative}\rangle$ to cell i.

Regardless of whether op_i reads cell j or not, op_j reads cell i after the last time op_i writes $\langle v, \mathsf{tentative}\rangle$ to cell i. The last such read of cell i by op_j occurs after op_j writes $\langle v, \mathsf{tentative}\rangle$ to cell j for the last time. Since this is the last such write and op_j writes $\langle v, \mathsf{final}\rangle$ to cell j, this value can

only change to be $\langle v, \mathsf{final}\rangle$. This read returns either $\langle v, \mathsf{tentative}\rangle$ or $\langle v, \mathsf{final}\rangle$ written by op_i to cell i. This is because op_i already performed its last write of $\langle v, \mathsf{tentative}\rangle$ to cell i, and this value can only change to $\langle v, \mathsf{final}\rangle$. Furthermore, by our initial assumption, once $\langle v, \mathsf{final}\rangle$ is written to cell i, it remains there until the write of $\langle v, \mathsf{final}\rangle$ to cell j.

If op_j reads $\langle v, \mathsf{final}\rangle$ or if $i - \mathsf{h}(v) < j - \mathsf{h}(v)$, op_j deletes its own value. Otherwise (op_j reads $\langle v, \mathsf{tentative}\rangle$), op_j tries to delete the copy in cell i. However, this attempt is unsuccessful since op_i inserts its copy. Thus, op_j rereads the cell to obtain $\langle v, \mathsf{final}\rangle$, and deletes its own copy. Since we assume that the read of cell i occurs after the last write by op_j of $\langle v, \mathsf{tentative}\rangle$ to cell j, the value of cell j cannot subsequently change to $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$. Consequently, op_j succeeds in deleting its own copy.

In all cases, op_j deletes its own copy and does not write $\langle v, \mathsf{final}\rangle$, which is a contradiction. □

5.2 Linearizability

Let α be a finite execution. An insert operation may return ABORT if it completes a full scan of the hash table and does not find an available cell. We ignore insert operations that return ABORT, as they do not affect the sequential specification; thus, they can be linearized at any point, provided they respect the real-time order.

We construct the linearization π_v of the remaining insert, delete and lookup operations in α. Let $H(\alpha)|_v$ be the projection of the history $H(\alpha)$ onto key v; that is, the subsequence of operations in $H(\alpha)$ involving the input key v. Our proof proceeds by constructing independent linearizations π_v of $H(\alpha)|_v$, for each key v. This is sufficient since the locality theorem of linearizability allows to obtain the linearization of the whole execution by merging per-key linearizations while respecting real-time order.

For the rest of the linearizability proof, fix a key v.

By Lemma 4, at most one winner can fully insert its copy at any point in the execution. Only after a fully-inserted copy is deleted, a new copy can be crowned the new winner. This gives us a well-defined order for successful insert and delete operations. When a delete operation deletes a partially inserted key, a successful insert and delete happen simultaneously. When there is also a fully inserted copy of the key at the same time, we first place the delete and then the insert. This ensures that the ordering ends with a successful insert at such a point. Otherwise, the insert is placed before the delete.

Ordering unsuccessful insert and delete operations, as well as lookup operations, is more challenging. When a lookup or insert operation finds a partially inserted copy of the key, there are three possibilities: (a) the copy is eventually fully inserted, (b) the copy will be deleted because of a later full insertion, or (c) the copy will be deleted because of an earlier full insertion. If the future successful insertion is not yet linearized, there is a pending insert operation, which we can treat as successful in the linearization. Later, when the actual winner is determined, the operation's response is updated accordingly.

An operation *op* is a *mutator* if it is either an $\mathsf{insert}(v)$ operation whose copy is inserted, or a $\mathsf{delete}(v)$ operation that deletes a copy of some insert operation. In the latter case, this means that *op* overwrites $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ with DELETED or $\langle v, \mathsf{final}\rangle$ with TOMBSTONE. Let mop be the set of all mutator operations in α with key v.

Note that a mutator operation does not necessarily complete in α, however, if it does, it returns *true*. If an insert(v) observes a partial copy before a winner exists, all later operations should treat v as present. In the linearization, we let one ongoing insert serve as the successful insert, even if it

eventually returns *false*. This captures the forced-success behavior of unsuccessful inserts described in Section 4.1.

We begin by ordering the mutator operations (Section 5.2.1). Next (Section 5.2.2), we introduce *insert sequences* and prove key properties that will be used (in Section 5.2.3) to place non-mutator operations in between mutator operations.

5.2.1 Ordering the Mutator Operations

We construct the permutation μ_v of the mutator operations, by assigning a *linearization point* to each mutator operation *op*, which is a step in α, between the invocation and the response of *op*. Two mutator operations may be linearized at the same step, in which case they are ordered, by marking them with $\{1, 2\}$, namely, which of them is first and which is second.

If $iop \in \mathsf{mop}$ is an $\mathsf{insert}(v)$ operation, then a copy of *iop* is overwritten with $\langle v, \mathsf{final}\rangle$ or DELETED. Let *pt* be the point in the execution where this overwrite happens. If the copy of *iop* is overwritten with $\langle v, \mathsf{final}\rangle$, we define $lin(iop) = \langle pt, 1\rangle$. Otherwise, a copy of *iop* is overwritten with DELETED. If at *pt* there is a cell in the table containing $\langle v, \mathsf{final}\rangle$, we define $lin(iop) = \langle pt, 2\rangle$. Otherwise, at *pt* no cell in the table contains $\langle v, \mathsf{final}\rangle$ and we define $lin(iop) = \langle pt, 1\rangle$.

If $dop \in \mathsf{mop}$ is a $\mathsf{delete}(v)$ operation, then *dop* overwrites $\langle v, \mathsf{final}\rangle$ with TOMBSTONE, or $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ with DELETED. Let *pt* be the point in the execution where this overwrite happens. If *dop* overwrites $\langle v, \mathsf{final}\rangle$ with TOMBSTONE, we define $lin(dop) = \langle pt, 1\rangle$. Otherwise, *dop* overwrites $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$, or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ with DELETED. If there is a cell in the table containing $\langle v, \mathsf{final}\rangle$ at *pt*, we define $lin(dop) = \langle pt, 1\rangle$. Otherwise, at *pt* no cell in the table contains $\langle v, \mathsf{final}\rangle$ and we define $lin(dop) = \langle pt, 2\rangle$.

Note that at points where a copy is overwritten with DELETED we linearize both an $\mathsf{insert}(v)$ operation and a $\mathsf{delete}(v)$ operation. If during this overwrite there is a cell in the table containing $\langle v, \mathsf{final}\rangle$, we linearize the delete first and then the insert. Otherwise, we linearize them in reverse order, first the insert then the delete. This ensures that a key is considered to be in the table if and only if it is stored in a cell with the flag final. We formalize this invariant below in Lemma 5.

We say that $\langle pt_1, i_1\rangle$ *precedes* $\langle pt_2, i_2\rangle$ if either pt_1 precedes pt_2 in α, or $pt_1 = pt_2$ and $i_1 < i_2$. Let μ'_v be a permutation of the operations in mop, ordered such that op_1 precedes op_2 in μ'_v if and only if $lin(op_1)$ precedes $lin(op_2)$. All mutator operations, including those that did not return in α, are assigned the response *true* in μ'_v.

We next formalize the behavior described in Section 4.1, where an insert operation that observes a partial copy may force the key to be treated as present, even if no insert has yet completed. We construct μ_v by possibly extending μ'_v. If at the end of α: (1) there is a cell in the table containing $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$, (2) μ'_v does not end with an $\mathsf{insert}(v)$ operation, and (3) there is no mutator $\mathsf{insert}(v)$ operation with a linearization point that occurs at the last step of α, we append such an operation as follows. Note that, by the code, the $\mathsf{insert}(v)$ operation this copy belongs to is pending. Among all pending $\mathsf{insert}(v)$ operations that write $\langle v, \mathsf{tentative}\rangle$ in α, let *op* be the one invoked the earliest. We obtain μ_v by appending *op* to μ'_v with response *true*. Furthermore, we add *op* to mop, and define its linearization point to be the last step in the execution. If these conditions are not met, we simply define $\mu_v = \mu'_v$.

Next, we show that, when ignoring the last insert possibly appended to μ_v, v is logically in the table if and only if the table contains $\langle v, \mathsf{final}\rangle$.

Lemma 5. *μ'_v ends with $\mathsf{insert}(v)$ if and only if some cell contains $\langle v, \mathsf{final}\rangle$ after α.*

Proof. We prove the lemma by induction on the number of steps in an execution prefix α' that either overwrite $\langle v, \mathsf{tentative}\rangle$ with $\langle v, \mathsf{final}\rangle$, or overwrite $\langle v, \mathsf{final}\rangle$ with TOMBSTONE, or overwrite $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ with DELETED. We note that this suffices since these *overwrite steps* are the only steps that append an operation to μ'_v. The base case, when no such steps appear in α', is trivial.

Assume the lemma holds for an execution prefix α', and consider the possible cases.

If $\langle v, \mathsf{tentative}\rangle$ *is overwritten with* $\langle v, \mathsf{final}\rangle$, then $\mathsf{insert}(v)$ is appended to μ'_v. Since $\langle v, \mathsf{final}\rangle$ is written, there is a cell containing $\langle v, \mathsf{final}\rangle$.

If $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ *or* $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ *are overwritten with* DELETED, then an $\mathsf{insert}(v)$ and a $\mathsf{delete}(v)$ are appended to μ'_v. If at the end of α' there is a cell containing $\langle v, \mathsf{final}\rangle$, $\mathsf{delete}(v)$ is appended first followed by $\mathsf{insert}(v)$. Since the $\langle v, \mathsf{final}\rangle$ value is not overwritten, until the next overwrite, a cell containing $\langle v, \mathsf{final}\rangle$ remains. If at the end of α' there is no cell containing $\langle v, \mathsf{final}\rangle$, $\mathsf{insert}(v)$ is appended first and then $\mathsf{delete}(v)$. Since no $\langle v, \mathsf{final}\rangle$ value is written, until the next overwrite, no cell contains $\langle v, \mathsf{final}\rangle$.

If $\langle v, \mathsf{final}\rangle$ *is overwritten with* TOMBSTONE, then $\mathsf{delete}(v)$ is appended to μ'_v. Since a $\langle v, \mathsf{final}\rangle$ value is deleted, and by Lemma 4, no other cell also contains $\langle v, \mathsf{final}\rangle$, no cell contains $\langle v, \mathsf{final}\rangle$. □

Lemma 6. *μ_v respects the sequential specification of a dictionary.*

Proof. Consider first μ'_v and assume by way of contradiction, that it does not respect the sequential specification of a dictionary. Let *op* be the first operation in μ'_v that violates the sequential specification. If *op* is an $\mathsf{insert}(v)$, then it appears after another $\mathsf{insert}(v)$. If *op* is a $\mathsf{delete}(v)$, then it is not preceded by an $\mathsf{insert}(v)$. Consider the overwrite in α that determines the placement of *op*.

If a $\langle v, \mathsf{tentative}\rangle$ *is overwritten by* $\langle v, \mathsf{final}\rangle$, then *op* is an $\mathsf{insert}(v)$. By Lemma 4, no cell contains $\langle v, \mathsf{final}\rangle$ before this overwrite, and by Lemma 5, μ'_v constructed up until before this overwrite, does not end with an $\mathsf{insert}(v)$, which is a contradiction.

If a $\langle v, \mathsf{final}\rangle$ *is overwritten with* TOMBSTONE, then *op* is a $\mathsf{delete}(v)$. By Lemma 5, μ'_v constructed up until before this overwrite, ends with an $\mathsf{insert}(v)$, which is a contradiction.

Finally, assume $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ *or* $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ *are overwritten by* DELETED. If there is a cell containing $\langle v, \mathsf{final}\rangle$ before this overwrite, then by Lemma 5, μ'_v constructed up until before this overwrite, ends with an $\mathsf{insert}(v)$. Thus, appending $\mathsf{delete}(v)$ and then $\mathsf{insert}(v)$ does not violate the sequential specification. Otherwise, no cell contains $\langle v, \mathsf{final}\rangle$ before this overwrite, and by Lemma 5, μ'_v constructed up until before this overwrite, does not end with an $\mathsf{insert}(v)$. Thus, appending $\mathsf{insert}(v)$ and then $\mathsf{delete}(v)$ does not violate the sequential specification.

Therefore, μ'_v respects the sequential specification.

Finally, assume that an $\mathsf{insert}(v)$ is appended to μ'_v to obtain μ_v. This happens only if μ'_v does not end with an $\mathsf{insert}(v)$, and therefore, appending the $\mathsf{insert}(v)$ obeys the sequential specification. □

5.2.2 Insert Sequences

A copy of an insert operation is either still present in the table, inserted, or is deleted because of a different copy of v. This yields a sequence of insert operations where a copy of each operation is deleted due to the next one in the sequence; the sequence ends when a copy of the final insert operation is successfully inserted.

A copy of an $\mathsf{insert}(v)$ operation op_1 is deleted because of an $\mathsf{insert}(v)$ operation op_2, denoted $op_2 \rightsquigarrow_{\mathsf{del}} op_1$, if either (1) op_2 overwrites a copy of op_1 with COLLIDED, or (2) op_1 overwrites its

own copy with TOMBSTONE after finding a (possibly inserted) copy of op_2. By the code, op_2 writes its first copy before op_1's copy is deleted.

Lemma 7 shows a stronger property: a copy cannot be deleted by an insert operation whose copy has already been deleted. The details of the next proof are the main place where the correctness proof differs between the LL/SC and the CAS versions of the algorithm.

Lemma 7. *Let op_1 and op_2 be two* insert(v) *operations such that $op_2 \rightsquigarrow_{\mathsf{del}} op_1$. Then either a copy of op_2 and the last copy of op_1 are both present in the table at the same time, or the last copy of op_1 is a $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ value and a copy of op_2 and the second-to-last copy of op_1 are both present in the table at the same time.*

Proof for the LL/SC *version.* If op_1 deletes its own copy, it does so after reading a copy of op_2. Consider the last such read by op_1, then op_1's next step is to overwrite its copy with TOMBSTONE. Assume that the copy of op_1 that is in the table during this last read is overwritten before a copy is deleted. Then, as op_1's copy is not deleted yet, and this overwrite is not performed by op_1, it can only be overwritten with $\langle v, \mathsf{revalidate}\rangle$. This copy cannot be overwritten until op_1's next step. However, $\langle v, \mathsf{revalidate}\rangle$ cannot be overwritten with TOMBSTONE, in contradiction.

If op_2 deletes a copy of op_1, then it begins by reading op_1's copy with an LL. Then, it reads its own cell and gets $\langle v, \mathsf{tentative}\rangle$. Lastly, it overwrites op_1's copy with an SC. If the SC succeeds, then it overwrites the same copy that it initially read with the LL. Therefore, this copy is the last copy of op_1. In between, it read its own copy, ensuring that a copy of op_2 is present in the table at the same time as op_1's last copy. □

Proof for the CAS *version.* Assume that op_1 deletes its own copy. Consider the last read of op_2's copy by op_1, then op_1's next step is to overwrite its copy with TOMBSTONE. If op_1's copy that is in the table during this last read is overwritten, it must be with $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$. This is because op_1's copy is not deleted yet, and this overwrite is not performed by op_1. We already showed for the LL/SC version that an overwrite with $\langle v, \mathsf{revalidate}\rangle$ is not possible, as the overwrite with TOMBSTONE must fail. Assume that the copy is overwritten with $\langle\langle v, *\rangle, \mathsf{marked}\rangle$. Since op_1 deletes its own copy, only op_1 can overwrite this value. Any other overwrite would mean that some other operation deleted this copy, which cannot happen. Thus, op_1 overwrites this copy with TOMBSTONE, and we get that op_2's copy is present in the table at the same time as op_1's second-to-last copy.

If op_2 deletes a copy of op_1, it first overwrites op_1's copy with $\langle\langle v, i\rangle, \mathsf{marked}\rangle$, where i is op_2's cell. Then, it reads $\langle v, \mathsf{tentative}\rangle$ from its own cell. Finally, it overwrites the value $\langle\langle v, i\rangle, \mathsf{marked}\rangle$ in op_1's cell with COLLIDED. At any point during the execution, only a single insert operation owns a given cell. This means that only op_1 and op_2 can overwrite the $\langle\langle v, i\rangle, \mathsf{marked}\rangle$ copy of op_1. Since op_2 is the only operation that can write the cell number i to the table, the value $\langle\langle v, i\rangle, \mathsf{marked}\rangle$ written by op_2 is the same one overwritten with COLLIDED, and this is the last copy of operation op_1. In between, op_2 read its own copy, ensuring that a copy of op_2 is present in the table at the same time as op_1's last copy. □

An *insert sequence* $op_k \rightsquigarrow_{\mathsf{del}} \ldots \rightsquigarrow_{\mathsf{del}} op_1$, $k \geq 1$, *has ended* if a copy of operation op_k is inserted. The insert sequence is *maximal* if it has ended or if a copy of operation op_k is still present in the table at the end of α, that is, its final copy is neither deleted nor inserted. This is denoted by $op_k \rightsquigarrow^*_{\mathsf{del}} op_1$.

Let c_j be the cell of op_j, $1 \leq j \leq k$. We next show that a copy is deleted only if its cell is further from the start of the probe sequence than the cell of the deleting operation (with the possible exception of the final deletion).

Lemma 8. *Consider a maximal insert sequence $op_k \rightsquigarrow_{\mathsf{del}} \ldots \rightsquigarrow_{\mathsf{del}} op_1$, $k \geq 1$. If $1 \leq j < k-1$, then $(c_{j+1} - \mathsf{h}(v)) < (c_j - \mathsf{h}(v))$. Furthermore, if the sequence has not ended, or if a copy of operation op_k is overwritten with* DELETED*, then $(c_k - \mathsf{h}(v)) < (c_{k-1} - \mathsf{h}(v))$.*

Proof. Note that no copy of op_{j+1} is overwritten with $\langle v, \mathsf{final}\rangle$. By the code and since $op_{j+1} \rightsquigarrow_{\mathsf{del}} op_j$, a copy of op_j is deleted after op_j finds a copy closer to the start of the probe sequence, or op_{j+1} deletes a copy further from the start. Therefore, $(c_{j+1} - \mathsf{h}(v)) < (c_j - \mathsf{h}(v))$, as needed. If the sequence has not ended, or if the copy of op_k is overwritten with DELETED, a copy of op_{k-1} cannot be deleted after op_{k-1} finds an inserted copy of op_k. As before, c_k is closer to the start of v's probe sequence than c_{k-1}. □

The next lemma is a key property of an insert sequence.

Lemma 9. *Consider a maximal insert sequence $op_k \rightsquigarrow_{\mathsf{del}} \ldots \rightsquigarrow_{\mathsf{del}} op_1$, $k > 1$. If a copy of operation op_k is not inserted, then among all operations op_j that have already written a copy to the table, no copy of the operation with the minimal distance $(c_j - \mathsf{h}(v))$ is deleted.*

Proof. The proof is by induction on the number of writes of the first copies of $op_1, \ldots op_k$, before the copy of op_k is inserted. For the base case, note that the first $\langle v, \mathsf{tentative}\rangle$ copy is deleted only because of a copy that is written after it. Thus, until the next write, the cell contains either $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$, or a copy of the operation is inserted, implying that op_k's copy is inserted.

For the induction step, assume that the lemma holds until the j-th write and that the copy of op_k is not overwritten with $\langle v, \mathsf{final}\rangle$ or DELETED. Consider the next, jth write of a first copy. By Lemma 8, a copy is deleted due to an $\mathsf{insert}(v)$ operation only if it is further from the start of the probe sequence than the copy of the deleting operation. Therefore, the copy residing in the cell closest to the start is deleted only because of a copy that is written after it. In the next write, the cell contains either $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$, or a copy of the operation is inserted, implying that op_k's copy is inserted. □

5.2.3 Placing the Non-Mutator Operations

We can now construct the permutation π_v by going through the rest of the completed operations, in the order in which they are invoked in the execution, and inserting them between the mutator operations, already placed in μ_v. We place each such operation *op immediately* before or after a mutator operation op'; in the latter case, we place *op* immediately before the next mutator operation that follows op' in μ_v, or at the end of the permutation if no such operation exists. This strategy ensures that operations placed after the same mutator operation respect the real-time order among them.

Let nop denote the set of the non-mutator operations that complete in α, and it has two subsets: nop^+ are the operations that *find the key*, that is, $\mathsf{insert}(v)$ operations that return *false* and $\mathsf{lookup}(v)$ operations that return *true*, while nop^- are the operations that *do not find the key*, that is, $\mathsf{delete}(v)$ and $\mathsf{lookup}(v)$ operations that return *false*.

We place these non-mutator operations by examining where the mutator operations are linearized. Specifically, for an operation in nop^+ (respectively, nop^-), we search for the earliest valid placement between the last mutator linearized before the operation begins and the first mutator linearized after it ends, such that v is present (respectively, absent) according to the sequential specification.

We start with the operations in nop^+. If there is a cell containing $\langle v, \mathsf{final}\rangle$ when an operation *op* in nop^+ is invoked, we place *op* in π_v after the mutator operation with the latest linearization point occurring before *op*'s invocation. If there is a mutator $\mathsf{insert}(v)$ operation with a linearization point occurring between *op*'s invocation and response, we place *op* after the earliest such point. If none of these two cases hold, we prove that there is an $\mathsf{insert}(v)$ in mop whose linearization point occurs after *op*'s response. Among these operations, let *iop* be the operation whose linearization point is the earliest; we place *op* in π_v after *iop*.

Lemma 10 shows that whenever a non-mutator insert or lookup operation returns after observing a key and attempting to replace it with a $\mathsf{revalidate}$ indication and the first two cases do not hold, there is an insert sequence whose final operation is ordered after it. This is used to show that the ordering of operations in nop^+ is well defined. The lemma also provides additional properties that are later used to prove that the ordering of these operations respects the real-time order.

Lemma 10. *Consider* $op \in \mathsf{nop}^+$*, and assume no cell contains* $\langle v, \mathsf{final}\rangle$ *when op is invoked, and there is no mutator* $\mathsf{insert}(v)$ *operation with a linearization point between the invocation and the response of op. Then, (a) there is a mutator* $\mathsf{insert}(v)$ *operation with a linearization point occurring after op returns.*
Furthermore, let $iop \in \mathsf{mop}$ *be the* $\mathsf{insert}(v)$ *operation with the earliest linearization point occurring after op's response. Then, (b) iop does not follow op, and (c) a cell containing key v is present in the table throughout the interval between op's response and lin*(iop)*.*

Proof. We pick an $\mathsf{insert}(v)$ operation *wop* that writes $\langle v, \mathsf{tentative}\rangle$ to the table, so that *op* is pending while a copy of *wop* is present in the table.

If *op* is a $\mathsf{lookup}(v)$ or an $\mathsf{insert}(v)$ where no write to the table occurs, then *op* returns after finding a cell containing the key v and possibly trying to overwrite it with $\langle v, \mathsf{revalidate}\rangle$. If *op* finds $\langle v, \mathsf{final}\rangle$ or $\langle v, \mathsf{revalidate}\rangle$, let this value be a (possibly inserted) copy of operation *wop*. If *op* finds $\langle v, \mathsf{tentative}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ and successfully overwrites it with $\langle v, \mathsf{revalidate}\rangle$, let both of these values be copies of operation *wop*. If the overwrite fails, it rereads the cell for the last time and finds a value with key v. Let this value be the (possibly inserted) copy of operation *wop*.

Otherwise, if *op* is an $\mathsf{insert}(v)$ that returns *false* after writing $\langle v, \mathsf{tentative}\rangle$, then let *wop* be operation *op* itself. Let op_k be the final operation in the maximal insert sequence starting with *wop* (i.e., $op_k \rightsquigarrow^*_{\mathsf{del}} wop$). We start by proving (a).

If no copy of op_k is ever inserted, a copy of v is present in the table at the end of α. By construction, either μ_v ends with an $\mathsf{insert}(v)$ operation, and so does π_v, or an $\mathsf{insert}(v)$ operation with a linearization point occurring at α's last step is placed at the end of π_v. Thus, there is a mutator $\mathsf{insert}(v)$ operation whose linearization point is after *op* returns.

Next, we assume that a copy of op_k is inserted. Assume, towards a contradiction, that the linearization point of op_k occurs before *op* returns. Since no $\mathsf{insert}(v)$ operation has a linearization point occurring between the invocation and the response of *op*, the linearization point by op_k occurs before *op*'s invocation. In other words, a copy of op_k is inserted before *op*'s invocation. Since no cell contains $\langle v, \mathsf{final}\rangle$ when *op* is invoked, op_k's copy is deleted before the invocation of *op*.

We show that op reads the cell of op_k in its forward scan before the write of the penultimate copy of op_k. However, since no new copy is written after a copy in inserted, this contradicts that a copy of op_k is inserted before the invocation of op.

First, we show that the read of op from the cell of iop, in its forward scan, precedes the write of the penultimate copy of iop (which is its only copy if iop has only one copy). If $op = iop$, this holds since op first completes the forward scan and then writes a copy for the first time. If $op \neq iop$ and it reads $\langle v, \mathsf{revalidate}\rangle$ or successfully overwrites a copy with $\langle v, \mathsf{revalidate}\rangle$, then the $\langle v, \mathsf{revalidate}\rangle$ is overwritten with $\langle v, \mathsf{tentative}\rangle$ before iop's copy is either inserted or deleted.

If the overwrite of op with $\langle v, \mathsf{revalidate}\rangle$ fails, it rereads the cell and finds a value containing the key v. This value cannot be $\langle v, \mathsf{final}\rangle$. This is because no cell contains $\langle v, \mathsf{final}\rangle$ when op is invoked and no insertion of v occurs before op returns. So, this value is either $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$ or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$. Then, either another copy of the same operation is written to the cell, or this copy is deleted and only then the first copy of iop is written to this cell.

Assume that for some j, $2 \leq j \leq k$, the write of the penultimate copy of op_j precedes the read of the cell of op_j in the forward scan of op; assume j is the first such operation in the insert sequence. Since $op_j \rightsquigarrow_{\mathsf{del}} op_{j-1}$, by Lemma 7, a copy of op_j is in the table after the write of the penultimate copy of op_{j-1}. Thus, no copy of op_j is deleted right after op reads the cell of op_{j-1}.

If $j = k$, this immediately leads to a contradiction. Otherwise, by Lemma 8, c_j is closer to the start of the probe sequence than c_{j-1}. Thus, in its forward scan, op first reads c_j and then c_{j-1}. So, the write of the penultimate copy of op_j precedes the read of c_j in the forward scan of op, and no copy of op_j is deleted before this read. Then, op encounters a copy of op_j in its forward scan. Since $j \neq k$, no copy of op_j is inserted. Consequently, since op attempts to overwrite this copy with $\langle v, \mathsf{revalidate}\rangle$, it follows that either two additional copies of op_j are written to the cell, or a copy of op_j is deleted before op continues to the next cell. Both scenarios yield a contradiction.

To prove (b), assume towards a contradiction that iop follows op. Therefore, since a copy of wop is written before op returns, iop is invoked after a copy of wop is first written. By (a), a copy of op_k is inserted after op's response, or not inserted at all. Thus, either $iop = op_k$ or a copy of iop is inserted before a copy of op_k is inserted. In both cases, the backward scan of iop starts after a copy of wop is written and ends before a copy of op_k is inserted.

Since $op_k \rightsquigarrow^*_{\mathsf{del}} wop$, Lemma 9 implies that after a copy of operation wop is first written and until a copy of operation op_k is inserted, the key v always appears in the table and can move only to a cell closer to the start of the probe sequence. This implies that iop finds the key v in its initial backward scan, which contradicts the fact that it is a mutator operation.

Finally, we prove (c). By (a), a copy of op_k is inserted after op's response, or not inserted at all. Thus, either $iop = op_k$ or a copy of iop is inserted before a copy of op_k is inserted. Since $op_k \rightsquigarrow^*_{\mathsf{del}} wop$, by Lemma 9, between the point where a copy of operation wop is first written and until a copy of operation op_k is inserted, the key v always appears in the table. This implies that throughout the interval between op's response and until a copy of operation op_k is inserted, there is a cell containing a copy of v belonging to one of the operations in the insert sequence. Since the interval between op's response and $lin(iop)$ is contained in this interval, the claim follows. □

Lemma 11. *The linearization of* nop^+ *respects the sequential specification of a dictionary.*

Proof. We prove that every operation op in nop^+ is placed immediately after a mutator $\mathsf{insert}(v)$ operation in π_v.

If there is a cell containing $\langle v, \mathsf{final}\rangle$ when *op* is invoked, then by Lemma 5, μ'_v constructed up to this point ends with an $\mathsf{insert}(v)$ operation. Thus, the last mutator operation placed up until this point in μ_v, and consequently also in π_v, is $\mathsf{insert}(v)$ and *op* is placed after it in π_v.

If there is a mutator $\mathsf{insert}(v)$ operation whose linearization point in between the invocation and the response of *op*, then *op* is placed after a mutator $\mathsf{insert}(v)$ operation.

Finally, if neither of these two cases holds, by Lemma 10(a), there is a mutator $\mathsf{insert}(v)$ operation with a linearization point occurring after *op* returns, and *op* is placed after the one with the earliest linearization point. □

Consider now an operation *op* in nop^-. If no cell contains the key v when *op* is invoked, or no copy of v is deleted between its invocation and response, we place *op* in π_v before the next mutator operation whose linearization point is after the invocation of *op*. If the first case does not hold, we prove that there is a $\mathsf{delete}(v)$ in mop whose linearization point occurs between *op*'s invocation and response. Hence, if neither of these conditions holds, let *dop* in mop be the $\mathsf{delete}(v)$ operation with the earliest linearization point between the invocation and response of operation *op*. We place *op* after *dop* in π_v.

Lemma 12. *Let $op \in \mathsf{nop}^-$. Suppose that when op is invoked, the table contains a (possibly inserted) copy of v belonging to operation op_1. Let $op_k \rightsquigarrow^*_{\mathsf{del}} op_1$, then a copy of operation op_k is inserted and deleted before op returns.*

Proof. First, assume that when *op* is invoked, the table contains a $\langle v, \mathsf{final}\rangle$ copy of op_1. Suppose for the sake of contradiction that this value is not deleted before *op* returns. According to the algorithm, *op* should encounter this value. If *op* is a $\mathsf{lookup}(v)$ operation, it returns *true*. If *op* is a $\mathsf{delete}(v)$ operation, it either successfully overwrites this value with TOMBSTONE and returns *true*, or fails and returns *false*. In the latter case, the failure implies that the $\langle v, \mathsf{final}\rangle$ value is overwritten with TOMBSTONE. In all cases, this contradicts the assumption that the value is not deleted.

Next, assume that when *op* is invoked, the table contains a copy of op_1 with value $\langle v, \mathsf{tentative}\rangle$, $\langle v, \mathsf{revalidate}\rangle$, or $\langle\langle v, *\rangle, \mathsf{marked}\rangle$.

If a copy of operation op_k is not deleted before *op* returns, by Lemma 9, there is always a copy of key v in the table, during both the forward and backward scans of the operation. Furthermore, if a copy of v is deleted, there is a copy of v in a cell closer to the start of the probe sequence. Hence, a copy of v is found during the backward scan of *op*.

If *op* is a $\mathsf{lookup}(v)$ operation, this implies that it returns *true*.

If *op* is a $\mathsf{delete}(v)$ operation, the copy cannot move closer to the start once it reaches the beginning of the probe sequence. There, *op* finds it, successfully overwrites it with DELETED, and returns *true*. However, this contradicts the assumption that *op* returns *false*. Hence, a copy of operation op_k is deleted before *op* returns.

Note that, by Lemma 9, until a copy of op_k is deleted, there is always a copy of v in the table that can move only to a cell closer to the start of the probe sequence.

If op_k's copy is overwritten with DELETED, the claim holds immediately. Otherwise, if it is overwritten with $\langle v, \mathsf{final}\rangle$ before *op* is invoked, then as shown in the first part of the proof, the copy is deleted before *op* returns.

Assume op_k's copy is overwritten with $\langle v, \mathsf{final}\rangle$ after *op*'s invocation. We consider three cases, and in all of them we show that the $\langle v, \mathsf{final}\rangle$ value of op_k is overwritten with TOMBSTONE before *op* returns. Note that it is enough to show that cell c_k is read in one of the scans of *op* after op_k's

copy is first written to the table. Since op returns *false*, a copy of op_k is either not found or op tries to overwrite it and fails. Both cases imply that $\langle v, \mathsf{final}\rangle$ is overwritten with TOMBSTONE.

Case 1: $c_k - \mathsf{h}(v) < c_{k-1} - \mathsf{h}(v)$. This implies that c_k is the cell closest to the start of the probe sequence among all cells in the sequence. Then, cell c_k is read in the backward scan after op_k's copy is first written; otherwise, this contradicts Lemma 9.

Case 2: $c_k - \mathsf{h}(v) > c_1 - \mathsf{h}(v)$. This implies that c_k is the cell furthest from the start of the probe sequence among all cells in the sequence.

We show that op_k's copy is first written before op_1's copy is first written. Assume towards a contradiction that this does not hold, and consider the last write of $\langle v, \mathsf{tentative}\rangle$ by op_k. Note that by the initial assumption, this value is subsequently overwritten with $\langle v, \mathsf{final}\rangle$.

During the scan performed by op_k following this write, the operation encounters a copy of v in a cell closer to the start of the probe sequence. Since op_1's copy is already present while op_k's copy is not inserted yet, this follows from Lemma 9. However, this implies that op_k deletes its own copy, which contradicts the fact that this copy is eventually inserted.

Recall that op_1's copy is already written when op is invoked. Since we showed that op_k's copy is written before op_1's copy, it follows that op_k's copy is also present when op is invoked. Consequently, op_k's copy is written before cell c_k is read during op's forward scan.

Case 3: For some index $1 \leq j < k-1$, $c_{j+1} - \mathsf{h}(v) < c_k - \mathsf{h}(v) < c_j - \mathsf{h}(v)$. That is, op_{j+1}'s cell is closer to the start of the probe sequence than op_k's cell, and op_j's cell is further from it.

We show that op_k's copy is first written before any of the copies of operations op_ℓ, $j+1 \leq \ell \leq k-1$, are first written. If this is not the case, consider the last write of $\langle v, \mathsf{tentative}\rangle$ by op_k. Note that by the initial assumption, this value is subsequently overwritten with $\langle v, \mathsf{final}\rangle$.

During the scan performed by op_k following this write, the operation encounters a copy of v in a cell closer to the start of the probe sequence. Since, for some $j+1 \leq \ell \leq k-1$, op_ℓ's copy is already present while op_k's copy is not inserted yet, this follows from Lemma 9. However, this implies that op_k deletes its own copy, which contradicts the fact that this copy is eventually inserted.

Note that in a backward scan, first cell c_j is read, then cell c_k, and finally cell c_{j+1}. If in op's backward scan it does not return after reading cell c_j, by Lemma 9, either op_k's copy is inserted, or a closer cell c_ℓ, for $j+1 \leq \ell \leq k$, contains a copy of v. This follows from op_1's copy already being present in the table before this scan begins.

As op_k's copy is written before any of the copies of operations op_ℓ, $j+1 \leq \ell \leq k-1$, are first written, a copy of op_k is written in the cell c_k when c_k is read in op's backward scan. □

Lemma 13. *The linearization of* nop^- *respects the sequential specification of a dictionary.*

Proof. Let $op \in \mathsf{nop}^-$. If op is placed directly after a $\mathsf{delete}(v)$ operation, the lemma clearly holds. If no cell contains $\langle v, \mathsf{final}\rangle$ at op's invocation point, then by Lemma 5, μ'_v constructed up to this point does not end with an $\mathsf{insert}(v)$ operation. Thus, the next mutator operation placed after this point in μ_v, and consequently also in π_v, if exists, should be an $\mathsf{insert}(v)$ operation and op can be placed before it in π_v. If there is a cell containing $\langle v, \mathsf{final}\rangle$ at op's invocation point, by Lemma 12, this $\langle v, \mathsf{final}\rangle$ value is deleted before op returns. Thus, by construction, op is placed after a $\mathsf{delete}(v)$ operation. □

The proof of the next lemma shows that π_v preserves real-time order. Together with Lemmas 6, 11 and 13, it follows that π_v is a linearization of α restricted to operations with input v.

Lemma 14. *For any execution* α, π_v *is a valid linearization of* $H(\alpha)|_v$.

Proof. Lemmas 6, 11, and 13 imply that π_v respects the sequential specification. It remains to prove that π_v preserves the real-time order.

The real-time order of operations in mop is preserved in π_v, since mutator operations are placed in π_v at a point between their invocation and response. If a final $\mathsf{insert}(v)$ is appended to μ'_v, resulting in μ_v, this operation is pending. Thus, no operation in α follows it, so appending it to the end of μ'_v preserves the real-time order.

Next, we show that operations in nop are placed after a mutator operation whose linearization point is in the interval between the last linearization point before the operation's invocation and the earliest linearization point after it returns. For operations in nop^-, this holds by definition.

Let $op \in \mathsf{nop}^+$. If there is a cell containing $\langle v, \mathsf{final}\rangle$ when op is invoked, or there is a mutator $\mathsf{insert}(v)$ operation with a linearization point occurring between op's invocation and response, the claim holds by definition. If neither of the cases holds, op is placed after $iop \in mop$, the $\mathsf{insert}(v)$ operation with the earliest linearization point occurring after the response of op. We prove that no other mutator operation is linearized between the response of op and $lin(iop)$.

By the definition of iop, no mutator $\mathsf{insert}(v)$ operation other than iop is linearized within this interval. Consider a mutator $\mathsf{delete}(v)$ operation dop with a linearization point in this interval. Recall our assumption that at op's invocation, no cell contains $\langle v, \mathsf{final}\rangle$, and no $\mathsf{insert}(v)$ is linearized between op's invocation and response. Consequently, no cell contains $\langle v, \mathsf{final}\rangle$ during the interval between op's response and $lin(iop)$. Hence, there is an insert operation linearized at the same execution step as dop that precedes dop in the linearization order, and this $\mathsf{insert}(v)$ operation must be iop. Therefore, the linearization point of dop must fall outside this interval.

Since the placement of operations in nop^- is confined to the interval bounded by the latest linearization point prior to the operation's invocation and the operation's response, the real-time order is preserved between operations in mop and nop^-, as well as among operations in nop^-.

Operations in nop^+ are placed in an interval that includes the earliest linearization point following the operation's response. This preserves the real-time order between any two operations in nop^+. We next show this also holds for an operation in nop^+ and operations in mop or nop^-.

Let $op^+ \in \mathsf{nop}^+$. If there is a cell containing $\langle v, \mathsf{final}\rangle$ when op is invoked, or there is a mutator $\mathsf{insert}(v)$ operation with a linearization point occurring between op's invocation and response, the placement interval ends at op's response. Consequently, in this case, the real-time order is naturally preserved between op^+ and any operation in mop or nop^+.

We next consider the case where neither condition holds. Here, op^+ is placed after $iop \in \mathsf{mop}$, defined as the $\mathsf{insert}(v)$ operation with the earliest linearization point following the response of op^+. Since we showed that no other mutator operation is linearized between op^+'s response and $lin(iop)$, the only real-time order violation can be between op^+ and iop. Yet, Lemma 10(b) guarantees that iop does not follow op^+.

Let $op^- \in \mathsf{nop}^-$. If op^+ follows op^-, then the placement intervals do not intersect, and the real-time order is preserved.

Assume op^+ precedes op^-, but is placed after it. If $lin(iop)$ occurs before the invocation of op^-, op^- is placed after op^+. Since op^+ is placed after op^-, $lin(iop)$ occurs after the invocation of op^-.

The invocation of op^- falls within the interval between the response of op^+ and $lin(iop)$. Consequently, by Lemma 10(c), at the time of this invocation, the table contains a copy of v.

By Lemma 12, a copy of v is deleted between op^-'s invocation and response. Let $dop \in \mathsf{mop}$ be the $\mathsf{delete}(v)$ operation with the earliest linearization point that occurs between op^-'s invocation and response. op^- is placed after dop. We already showed that no other mutator operation is

linearized between the response of op^+ and $lin(iop)$. Hence, $lin(iop)$ precedes $lin(dop)$ in α, and op^+ is placed before op^- is π_v. □

5.3 Liveness

It is easy to see that lookup operations are wait-free. For delete(v) and insert(v) operations, the only cases in which these operations take infinitely many steps without returning are:

- For delete(v): the operation repeatedly attempts and fails to overwrite a tentative copy of v at a specific cell with DELETED;
- For insert(v): the operation writes a tentative copy into some cell, but is then forced to revalidate it infinitely often by signals from other processes.

In both cases, the operation attempts infinitely many times to modify a specific cell containing a copy of v. Thus, to prove lock-freedom, we show that this scenario arises only if infinitely many other operations complete.

Since lookup and delete operations modify a memory cell only once during their execution, their interference is naturally bounded. The difficult case is insert operations, for which we prove two lemmas: one bounds the interference from copies of other operations, and the other shows that an operation either rewrites its copy a bounded number of times or infinitely many operations complete.

Lemma 15. *An* insert(v) *operation op performs at most a bounded number of modifying instructions on cells containing a copy of v belonging to other operations.*

Proof. From the code, *op* modifies a cell containing a tentative copy *of a different operation* in the following situations: (1) *op* overwrites a tentative copy with $\langle v, \mathsf{revalidate}\rangle$, or (2) in the CAS version, *op* overwrites a tentative copy with $\langle\langle v, j\rangle, \mathsf{marked}\rangle$, or (3) in both the CAS and LL/SC versions, *op* overwrites a tentative copy with COLLIDED.

Case (1) can happen only once before *op* returns.

We show that *op* performs only a constant number of modifying instructions corresponding to cases (2) and (3), before either returning or moving to a state from which it returns with a finite number of steps. Assume towards contradiction this is not the case. Then, *op* overwrites a tentative copy with COLLIDED infinitely many times, or, in the CAS version, with $\langle\langle v, j\rangle, \mathsf{marked}\rangle$.

Assume *op* overwrites a tentative copy with COLLIDED infinitely many times. Once a tentative copy is overwritten with COLLIDED, a new tentative copy of the same operation is never written again. Thus, *op* is overwriting tentative copies of distinct operations.

Consider the sequence of operations whose copies are overwritten by *op*. Since this sequence is infinite, we can identify an operation op' in this sequence that is invoked after *op* has already written its own initial tentative copy. Before *op* overwrites a tentative copy with COLLIDED, it verifies that its own tentative copy has not been deleted; therefore, a tentative copy of *op* remains present in the table throughout the execution interval of op'.

Before writing a tentative copy, op' first scans a run starting from $h(v)$. By Proposition 3, during this scan, op' encounters a tentative copy of *op*. Since no copy of *op* is ever deleted, op' should return *false* immediately, without writing a new tentative copy to the table. This contradicts the assumption that op' writes a tentative copy, which *op* later overwrites.

Next, consider the case where op overwrites a tentative copy with $\langle\langle v, j\rangle, \mathsf{marked}\rangle$ infinitely many times. If op overwrites tentative copies belonging to infinitely many distinct operations, the proof proceeds as in the previous case. Otherwise, op repeatedly overwrites a tentative copy of the same operation op'. This implies that op' repeatedly rewrites its cell to $\langle v, \mathsf{tentative}\rangle$ (potentially after being reset to $\langle v, \mathsf{revalidate}\rangle$).

Since op tries to delete a copy of op', op's tentative copy is closer to the start of the probe sequence of v than the tentative copy of op'. Furthermore, because op verifies the existence of its own copy before performing the overwrite, the tentative copy of op remains present in the table throughout the execution interval of op'. Each time op' rewrites the value $\langle v, \mathsf{tentative}\rangle$, it scans a run again starting from $h(v)$. By Proposition 3, op' encounters the tentative copy of op during this scan. Since op's copy is closer to the start of the probe sequence, op' attempts to delete its own copy. This deletion attempt is unsuccessful, as we assumed infinitely many tentative copies of op' are written. This failure can only occur due to an intervention by a third operation op'' overwriting op''s tentative copy with $\langle v, \mathsf{revalidate}\rangle$.

As this occurs infinitely many times, we can choose an op'' invoked after op writes its first tentative copy. In op'''s forward scan, it encounters op's tentative copy (which is closer to the start of the probe sequence) before reaching op''s tentative copy. Consequently, since no copy of op is ever deleted, op'' returns immediately. This contradicts the claim that op'' reaches op' tentative copy and overwrites it with $\langle v, \mathsf{revalidate}\rangle$. □

Lemma 16. *An* $\mathsf{insert}(v)$ *operation op either performs at most a bounded number of modifying instructions on the cell containing its own copy, or infinitely many other operations return during the execution interval of op.*

Proof. Consider an $\mathsf{insert}(v)$ operation op on some key v. By the code, op modifies a cell containing *its own* tentative copy either when (1) op promotes its own copy to $\langle v, \mathsf{final}\rangle$, deletes its own copy by writing TOMBSTONE, or (2) op writes a new copy of $\langle v, \mathsf{tentative}\rangle$.

In case (1), op overwrites its tentative copy with one of these values only once before returning.

In case (2), op infinitely many times overwrites either $\langle v, \mathsf{revalidate}\rangle$ with $\langle v, \mathsf{tentative}\rangle$, or, in the CAS version, $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ with $\langle v, \mathsf{tentative}\rangle$.

If op overwrites $\langle v, \mathsf{revalidate}\rangle$ with $\langle v, \mathsf{tentative}\rangle$ infinitely many times, then since an $\mathsf{insert}(v)$ or $\mathsf{lookup}(v)$ operation overwrites a tentative copy with $\langle v, \mathsf{revalidate}\rangle$ at most once before returning, implying that infinitely many $\mathsf{insert}(v)$ or $\mathsf{lookup}(v)$ operations return.

Otherwise, op overwrites $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ with $\langle v, \mathsf{tentative}\rangle$ infinitely many times. By Lemma 15, any single operation overwrites a tentative copy with $\langle\langle v, *\rangle, \mathsf{marked}\rangle$ only a finite number of times before returning. Therefore, an infinite number of such operations return. □

Lemma 17. *For any key v, an operation on v either performs at most a bounded number of modifying instructions to cells containing v before returning, or infinitely many operations return during the execution interval of the operation.*

Proof. A $\mathsf{lookup}(v)$ overwrites a tentative copy with $\langle v, \mathsf{revalidate}\rangle$ only once before returning. A $\mathsf{delete}(v)$ overwrites a tentative copy with DELETED or an inserted copy with TOMBSTONE only once before returning. For $\mathsf{insert}(v)$, the lemma follows from Lemma 15 and Lemma 16. □

In particular, if an execution has infinitely many such modifying instructions on cells containing v, then infinitely many operations on v return. We will use this lemma to prove that delete and insert operations are lock-free.

Lemma 18. delete *is lock-free.*

Proof. Consider a delete(v) operation *op* that takes infinitely many steps without returning. Then *op* repeatedly reads $\langle v, \text{tentative}\rangle$ or $\langle v, \text{revalidate}\rangle$, and in the CAS version also $\langle\langle v, *\rangle, \text{marked}\rangle$, and fails to overwrite it with DELETED. Each such failure indicates that, between *op*'s read and its attempted write, another operation on v performed a modifying instruction on the cell. By Lemma 17, each operation on v can perform only a bounded number of modifying instructions before returning, or infinitely many other operations return. □

Lemma 19. insert *is lock-free.*

Proof. Consider an insert(v) operation *op* that takes infinitely many steps without returning.

If *op* never successfully writes $\langle v, \text{tentative}\rangle$, it returns after the forward or backward scan, or after traversing all m cells in an attempt to find an available slot.

Otherwise, *op* writes $\langle v, \text{tentative}\rangle$. If *op* takes an infinite number of steps without returning, then this value is overwritten infinitely many times with $\langle v, \text{revalidate}\rangle$, in the LL/SC version, and with $\langle v, \text{revalidate}\rangle$ or $\langle\langle v, *\rangle, \text{marked}\rangle$, in the CAS version. Each such overwrite is a modifying instruction to a cell containing v performed by some operation on v. By Lemma 17, each operation on v can perform only a bounded number of such modifying instructions before returning, except when infinitely many other operations return. Thus, an insert cannot take infinitely many steps without infinitely many operations completing. □

5.4 Amortized Step Complexity

In this section, we analyze the runtime of our algorithm under the simplifying assumption that at any point in time, there is at most one active insert operation for any given key. (Multiple concurrent operations on the same key are permitted, as long as only one of them is an insertion.) Without this assumption, the adversary can force the creation of tombstones (as demonstrated in Fig. 2), though we note that it can only deterministically force $O(n)$ tombstones in any given run in the hash table.

We recall that the classical analysis of Knuth [12] for sequential linear-probing hash tables shows that after any sequence of k insertions, if we represent the resulting load factor on the table by $k/m = 1 - 1/x$ for $x > 0$, then for any key v, the expected distance from $h(v)$ to the end of v's run is at most $O(x^2)$.

Mapping to a sequential execution. Knuth [12] did not address tombstones in his analysis (in fact, he addressed only insertions, but his analysis holds if deletions are implemented by reordering the remaining keys to take the empty place of the deleted key, rather than using tombstones). To address this gap we perform a pessimistic analysis that ignores the possibility that an inserted key will take the place of a previously deleted one: we assume that any inserted remains in the hash table, ignoring the possible re-use of tombstones that are left behind by successful deletions.

To map concurrent executions to sequential ones, we define the *effective insertion schedule* of an execution α. This schedule consists of the subsequence of insert operations in α that perform a write of $\langle *, \text{tentative}\rangle$, ordered by the time of that write. This implies the following:

Proposition 20. *Let α be an execution consisting only of* insert *operations, where no two concurrent operations have the same key. Then, the state of the table at the end of α is identical to the state after the sequential execution of the effective insertion schedule of α.*

Consider a sequential execution of all the insert operations in the batch. By Knuth's analysis the expected run length for any given key is $O(x^2)$. Crucially, this holds regardless of the execution order. Thus, the specific effective insertion schedule determined by the concurrent execution is irrelevant to the expected run length.

As in the sequential algorithm, our delete and lookup operations never occupy a cell that was previously empty, and therefore they can never extend an existing run or cause two adjacent runs to merge; only insert operations can increase the run length. By Proposition 20, provided there are no concurrent insert operations targeting the same key, the sequential analysis suffices to determine the expected run length. Consequently, the expected length of any scan over consecutive non-empty cells in the concurrent execution is also $O(x^2)$.

We define a *c-bounded fixed-workload scheduler* as an adversary that manages a batch of operations that is fixed in advance, before the hash function is chosen. The scheduler is worst-case, and determines the invocation order dynamically: whenever a process is free, the scheduler may choose *any* of the remaining operations in the batch to run next, subject to (a) a maximum *point contention* of c per key, that is, the maximum number of concurrent operations targeting the same key at any point in time is at most c; and (b) a maximum of one insert operation active on any given key at any time. Under this scheduling model, we establish the following bound on the step complexity, applicable to both the LL/SC and CAS versions of our algorithm.

Theorem 21. *Let m be the size of the table, let $x > 1$ be a real number, and fix any batch of operations that includes at most $(1 - 1/x)m$ insert operations. Under a c-bounded fixed-workload scheduler, the expected amortized step complexity per operation in both the LL/SC and CAS versions is $O(x^2 + c)$.*

Proof. We design a *charging scheme* that charges each step taken by process p to some operation, either its own operation or a different one. Recall that because concurrent insert operations operate on distinct keys, the state $\langle\langle *, * \rangle, \mathsf{marked}\rangle$ (required in the CAS version for colliding insertions) is never written. Consequently, the analyses of the LL/SC and CAS implementations coincide, and we treat them together.

Lookup operations: A lookup operation performs at most one forward and one backward scan, each traversing expected $O(x^2)$ cells, as established earlier. Each step of a scan takes a constant number of read and modify instructions, which are charged to the lookup itself.

Delete operations: A delete operation performs at most one forward and one backward scan. However, it may execute a super-constant number of instructions on a single cell if it repeatedly encounters a value $\langle v, \mathsf{tentative}\rangle$ or $\langle v, \mathsf{revalidate}\rangle$ and fails to update it.

- The initial read of a cell and any successful modify instructions are charged to the delete operation itself.
- If the delete operation reads a value that does not contain the key v (or contains $\langle v, \mathsf{final}\rangle$), it stops re-reading the cell and proceeds. This read is charged to the delete operation itself.
- Any subsequent pair of modify and read instructions representing a failed attempt to update the cell is charged to the *interfering* insert or lookup operation that wrote the value triggering the retry.

1. If the value is $\langle v, \mathsf{revalidate}\rangle$, the pair is charged to the operation that wrote this value.
2. If the value is $\langle v, \mathsf{tentative}\rangle$ and it is the first such value written by the current insert operation, the pair is charged to that insert operation.
3. If the value is $\langle v, \mathsf{tentative}\rangle$ but it overwrites a $\langle v, \mathsf{revalidate}\rangle$, the pair is charged to the operation that wrote the preceding $\langle v, \mathsf{revalidate}\rangle$.

A delete operation is charged expected $O(x^2)$ of its own steps. Steps are charged to other operations only when the delete operation re-reads a cell. Consequently, the operation that triggers this retry must be concurrent with the delete operation. An insert or lookup operation restarts at most one other operation (by writing $\langle v, \mathsf{revalidate}\rangle$ exactly once). Since there are at most c concurrent operations targeting key v, at most $c-1$ delete operations can read this $\langle v, \mathsf{revalidate}\rangle$ and charge the writer. Similarly, if $\langle v, \mathsf{revalidate}\rangle$ is subsequently overwritten by $\langle v, \mathsf{tentative}\rangle$, this write can cause at most $c-1$ additional concurrent delete operations to restart. Separately, an insert operation that writes $\langle v, \mathsf{tentative}\rangle$ is charged with the retries of at most $c-1$ concurrent delete operations that read this value. Thus, an insert or lookup is charged with at most $O(c)$ steps from concurrent delete operations.

Insert operations: An insert operation begins by searching for the key by performing at most one forward and one backward scan. If found, it terminates. If not, it performs a forward scan to find an empty slot. Since the table is not full, a slot is guaranteed to be found. This initial phase comprises at most three scans. With constant work per cell, the expected cost is $O(x^2)$, charged to the insert operation itself.

In the second phase, the insert repeatedly performs forward scans to finalize the insertion. If an insert operation attempts to delete its own copy, it succeeds in modifying the cell after a finite constant number of attempts, unless the cell is marked with revalidate. This holds because once the operation's cell transitions to a cleanup state (DELETED or COLLIDED), only the owner can modify it.

A forward scan is repeated only if a different insert or lookup restarts this operation.

- The first forward scan is charged to the insert operation itself.
- Any subsequent scans are charged to the operation that triggered the restart.

Since an insert or lookup restarts at most one other operation, it is charged with at most one additional forward scan of a different insert operation, adding expected $O(x^2)$ steps.

Each operation is charged with expected $O(x^2)$ steps for its own execution, at most $O(c)$ steps from interfering deletes, and at most $O(x^2)$ steps from a restarted insert. The total expected amortized step complexity is therefore $O(x^2 + c)$. □

6 Summary and Discussion

We present a lock-free linear probing hash table that minimizes memory overhead: it uses only 2 bits of metadata per entry with LL/SC, and $O(\min(\log m, \log n))$ bits using CAS. Our hash table supports wait-free lookups, and in the relatively uncontended scenario where keys are not inserted

more than once at the same time, it has roughly the same performance as sequential linear probing, up to an additive term of the point contention on any given key.

In the world of *sequential* data structures, much effort is devoted to designing *succinct data structures*, whose memory footprint matches the information-theoretic lower bound on the abstract data type they represent, up to lower-order terms. For hash tables (or rather, for dictionaries) such constructions have been known for over two decades, and the precise tradeoff between memory and query time has recently been completely resolved [4, 13]. For concurrent dictionaries, however, no succinct lock-free construction is known; to our knowledge, our construction appears to come closest, at $m(\lceil \log(U) \rceil + 2)$ bits in total (in the LL/SC version). Obtaining succinct lock-free dictionaries is an interesting open problem.

Acknowledgments: Hagit Attiya is supported by the Israel Science Foundation 22/1425 and 25/1849. Rotem Oshman and Noa Schiller are supported by NSF-BSF 2022699.

References

[1] James H. Anderson and Mark Moir. Universal constructions for multi-object operations. In *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '95, pages 184—193, New York, NY, USA, 1995. Association for Computing Machinery. doi:10.1145/224964.224985.

[2] Arm Ltd. *Arm Architecture Reference Manual for A-profile architecture*. Arm Ltd., 2026. Document ID: DDI 0487, Issue M.b. URL: https://developer.arm.com/documentation/ddi0487/.

[3] Hagit Attiya, Michael A. Bender, Martín Farach-Colton, Rotem Oshman, and Noa Schiller. History-independent concurrent hash tables. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1283—1294, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3717823.3718283.

[4] Michael A. Bender, Martín Farach-Colton, John Kuszmaul, William Kuszmaul, and Mingmou Liu. On the optimal time/space tradeoff for hash tables. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 1284–1297, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3519935.3519969.

[5] Guy E. Blelloch and Yuanhao Wei. LL/SC and Atomic Copy: Constant Time, Space Efficient Implementations Using Only Pointer-Width CAS. In Hagit Attiya, editor, *34th International Symposium on Distributed Computing (DISC 2020)*, volume 179 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:17, Dagstuhl, Germany, 2020. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.DISC.2020.5, doi:10.4230/LIPIcs.DISC.2020.5.

[6] Cliff Click. A lock-free wait-free hash table, 2007. Accessed on 2024-07-04. URL: http://www.stanford.edu/class/ee380/Abstracts/070221.

[7] Hui Gao, Jan Friso Groote, and Wim H. Hesselink. Lock-free dynamic hash tables with open addressing. *Distrib. Comput.*, 18(1):21—42, July 2005. doi:10.1007/s00446-004-0115-2.

[8] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming, Second Edition*. Elsevier, 2020. doi:10.1016/C2011-0-06993-4.

[9] Maurice Herlihy, Nir Shavit, and Moran Tzafrir. Hopscotch hashing. In *Proceedings of the 22nd International Symposium on Distributed Computing (DISC)*, pages 350—364, Berlin, Heidelberg, 2008. Springer-Verlag. doi:10.1007/978-3-540-87779-0_24.

[10] Amos Israeli and Lihu Rappoport. Disjoint-access-parallel implementations of strong shared memory primitives. In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '94, pages 151—-160, New York, NY, USA, 1994. Association for Computing Machinery. doi:10.1145/197917.198079.

[11] Prasad Jayanti and Srdjan Petrovic. Efficient and practical constructions of ll/sc variables. In *Proceedings of the Twenty-Second Annual Symposium on Principles of Distributed Computing*, PODC '03, pages 285—294, New York, NY, USA, 2003. Association for Computing Machinery. doi:10.1145/872035.872078.

[12] Donald E. Knuth. Notes on "open" addressing. Unpublished memorandum, July 1963.

[13] Tianxiao Li, Jingxun Liang, Huacheng Yu, and Renfei Zhou. Tight cell-probe lower bounds for dynamic succinct dictionaries. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1842–1862, 2023. doi:10.1109/FOCS57990.2023.00112.

[14] Tobias Maier, Peter Sanders, and Roman Dementiev. Concurrent hash tables: Fast and general(?)! *ACM Trans. Parallel Comput.*, 5(4), February 2019. doi:10.1145/3309206.

[15] Maged M. Michael. High performance dynamic lock-free hash tables and list-based sets. In *Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA)*, pages 73–82. ACM, 2002.

[16] Mark Moir. Practical implementations of non-blocking synchronization primitives. In *Proceedings of the Sixteenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '97, pages 219—228, New York, NY, USA, 1997. Association for Computing Machinery. doi:10.1145/259380.259442.

[17] Nhan Nguyen and Philippas Tsigas. Lock-free cuckoo hashing. In *2014 IEEE 34th International Conference on Distributed Computing Systems*, pages 627–636, 2014.

[18] Chris Purcell and Tim Harris. Non-blocking hashtables with open addressing. In *Proceedings of the 19th International Conference on Distributed Computing (DISC)*, pages 108—-121, Berlin, Heidelberg, 2005. Springer-Verlag. doi:10.1007/11561927_10.

[19] Ori Shalev and Nir Shavit. Split-ordered lists: Lock-free extensible hash tables. *Journal of the ACM*, 53(3):379–405, 2006.

[20] Julian Shun and Guy E. Blelloch. Phase-concurrent hash tables for determinism. In *Proceedings of the 26th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 96—107, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2612669.2612687.